

Stock Options

Frank Agyemang, Dr. Blaine Johnson, Dr. Allan Fritz, Addy Carrol, Dr. Reka Howard

2026-08-29

Reading in stocks with stock options

Stock Return Analysis: Fetch, Compute Returns, Plot, Decompose (Stock Options)

```
## Define tickers and date range

tickers <- c("GRAB", "FRMI", "CAAS", "IMRX", "ACDC", "OWLT",
            "DFDV", "TOUR", "ANNX", "FNGR", "PLUG", "KEEL",
            "TE", "NIO", "OPEN", "BTBT", "SPY")
start_date <- "2025-01-05"
end_date <- Sys.Date()

## Download price data (robust version)
safe_getSymbols_options <- function(ticker) {
  tryCatch({
    getSymbols(ticker, src = "yahoo", from = start_date, to = end_date, auto.assign = FALSE)
  }, error = function(e) {
    message("Failed to download: ", ticker, " -- ", conditionMessage(e))
    NULL
  })
}

price_data_options <- lapply(tickers, safe_getSymbols_options)
names(price_data_options) <- tickers

# Drop tickers that failed
failed_tickers <- names(price_data_options)[sapply(price_data_options, is.null)]
if (length(failed_tickers) > 0) {
  message("Skipped (failed to download): ", paste(failed_tickers, collapse = ", "))
}
price_data <- price_data_options[!sapply(price_data_options, is.null)]
tickers <- names(price_data) # update tickers to only the successful ones

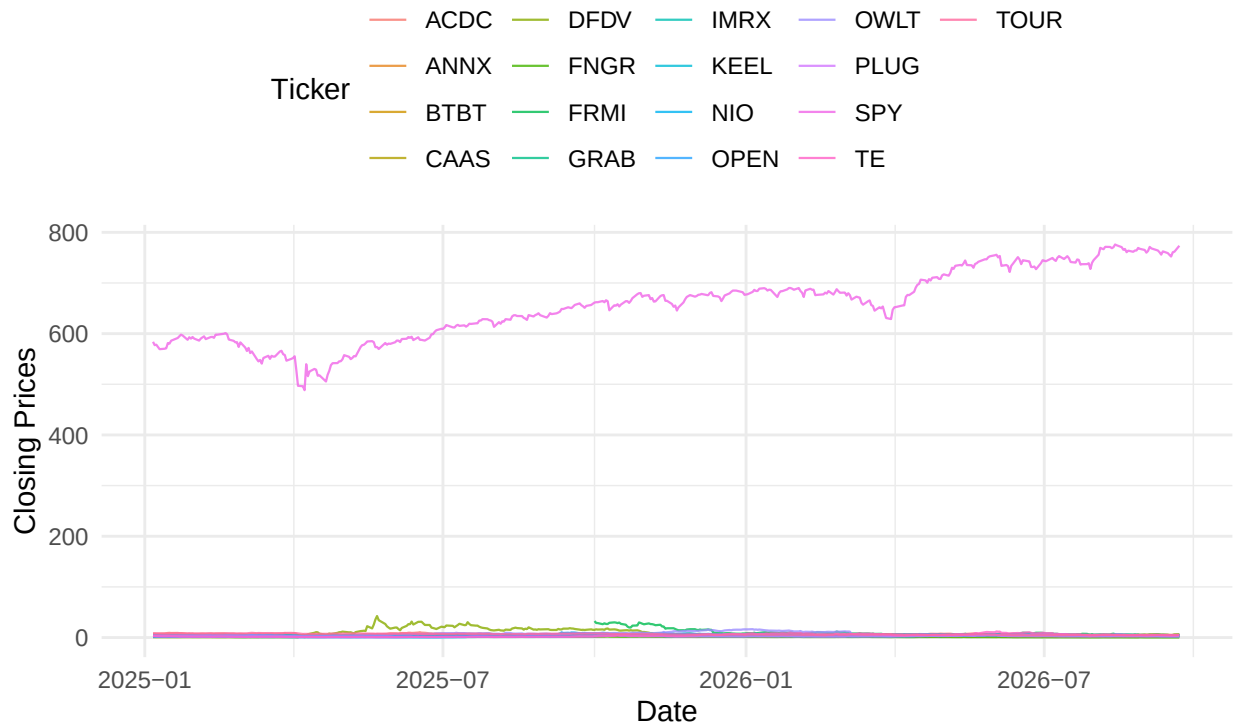
price_list <- lapply(price_data, Ad)
prices <- do.call(merge, price_list)
colnames(prices) <- tickers

## Plotting times series for stock prices
prices_df <- prices %>%
  fortify.zoo() %>% # converts xts to data frame with a Date column
  rename(Date = Index) %>%
  pivot_longer(-Date, names_to = "Ticker", values_to = "Price")
```

```
ggplot(prices_df, aes(x = Date, y = Price, color = Ticker)) +
  geom_line(alpha = 0.8, linewidth = 0.4) +
  labs(title = "Daily Closing Prices",
       subtitle = paste(tickers, collapse = ", "),
       x = "Date", y = "Closing Prices") +
  theme_minimal() +
  theme(legend.position = "top")
```

Daily Closing Prices

GRAB, FRMI, CAAS, IMRX, ACDC, OWLT, DFDV, TOUR, ANNX, FNGR, PLUG, KEEL, 1



```
## Calculate daily returns
# Log returns (Used here)
returns_log <- na.omit(Return.calculate(prices, method = "log"))

## Superimposed time series plot of returns
returns_df <- returns_log %>%
  fortify.zoo() %>%      # converts xts to data frame with a Date column
  rename(Date = Index) %>%
  pivot_longer(-Date, names_to = "Ticker", values_to = "Return")

daily_ret <- ggplot(returns_df, aes(x = Date, y = Return, color = Ticker)) +
  geom_line(alpha = 0.8, linewidth = 0.4) +
  labs(title = "Daily Log Returns",
       subtitle = paste(tickers, collapse = ", "),
       x = "Date", y = "Log Return") +
  theme_minimal() +
  theme(legend.position = "top")
```

```

# Save Plot
ggsave(filename = "Daily Returns.png",
        plot = daily_ret,
        width = 8,
        height = 6, units = "in", dpi = 300)

## Decomposition: trend, seasonality, cyclical, residual
## This requires the `mFilter` package

decompose_stock <- function(ret_xts, ticker, frequency = 21) {
  # frequency = 21 treats a trading month as the "seasonal" period;
  # use 5 for a trading week, or 252 for an annual cycle instead.
  ts_data <- ts(as.numeric(ret_xts), frequency = frequency)

  stl_fit <- stl(ts_data, s.window = "periodic", robust = TRUE)
  comp <- as.data.frame(stl_fit$time.series) # seasonal, trend, remainder
  # Further split the STL trend into long-run trend + cycle via HP filter
  hp_fit <- hpfilter(comp$trend, freq = 14400) # 14400 is a common daily-data lambda
  comp$long_run_trend <- as.numeric(hp_fit$trend)
  comp$cycle <- as.numeric(hp_fit$cycle)

  comp$Date <- index(ret_xts)
  comp$Ticker <- ticker
  comp
}

decomp_list <- lapply(tickers, function(t) decompose_stock(returns_log[, t], t))
decomp_all <- bind_rows(decomp_list)

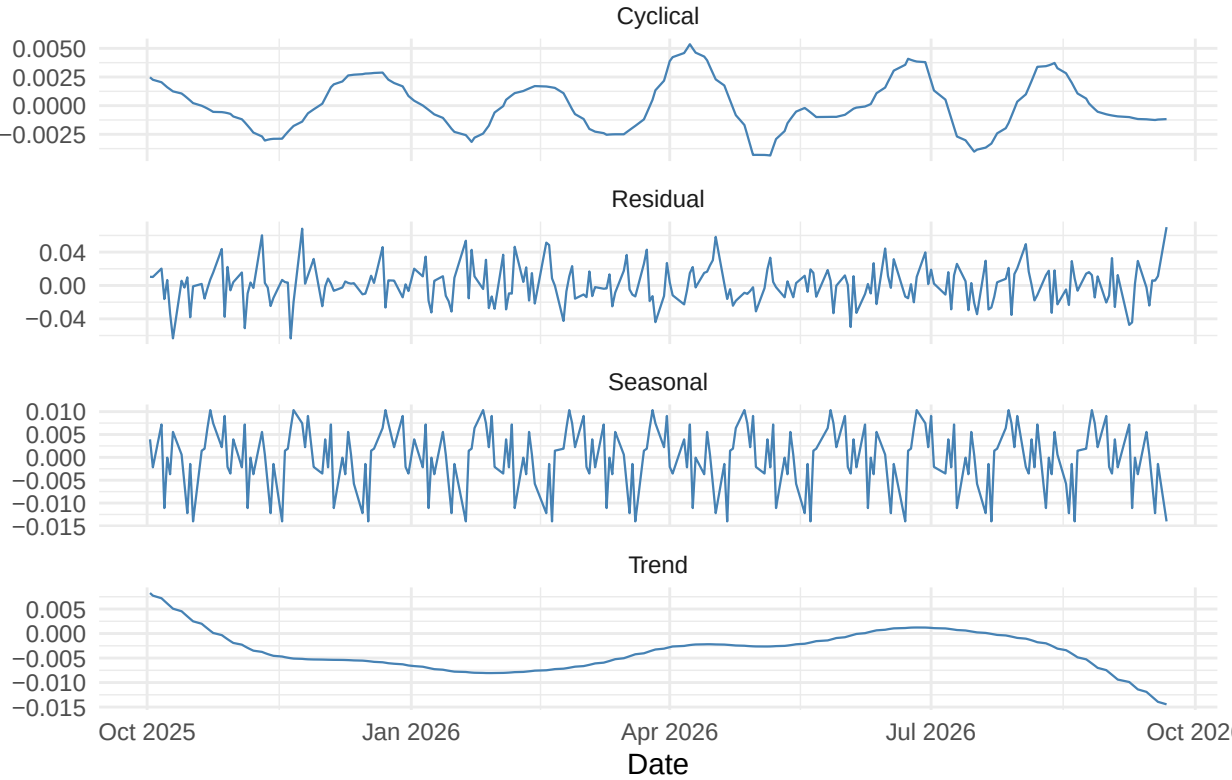
## Plot each stock's decomposition
plot_decomposition <- function(df, ticker) {
  df_long <- df %>%
    filter(Ticker == ticker) %>%
    select(Date, seasonal, long_run_trend, cycle, remainder) %>%
    rename(Trend = long_run_trend, Seasonal = seasonal,
           Cyclical = cycle, Residual = remainder) %>%
    pivot_longer(-Date, names_to = "Component", values_to = "Value")

  ggplot(df_long, aes(x = Date, y = Value)) +
    geom_line(color = "steelblue", linewidth = 0.4) +
    facet_wrap(~ Component, scales = "free_y", ncol = 1) +
    labs(title = paste("Decomposition of Daily Returns:", ticker),
         x = "Date", y = NULL) +
    theme_minimal()
}

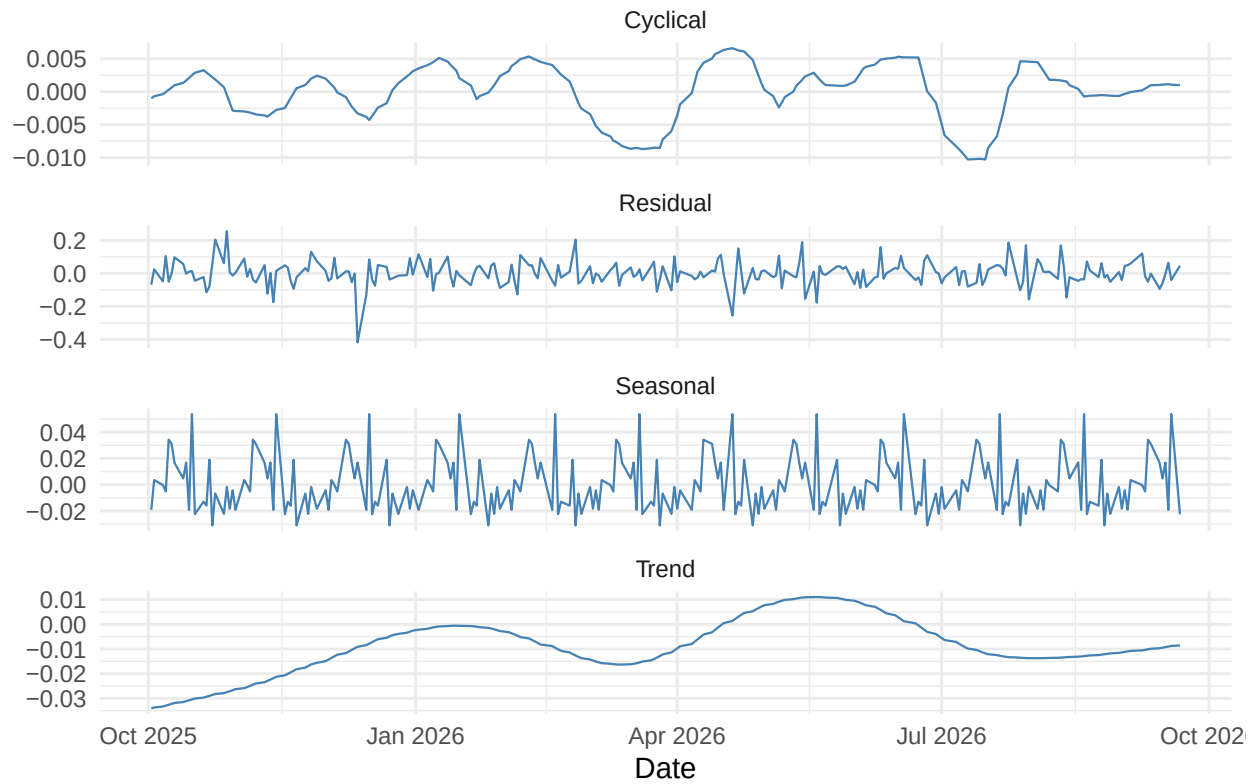
# Example: plot decomposition for each ticker
for (t in tickers) {
  print(plot_decomposition(decomp_all, t))
}

```

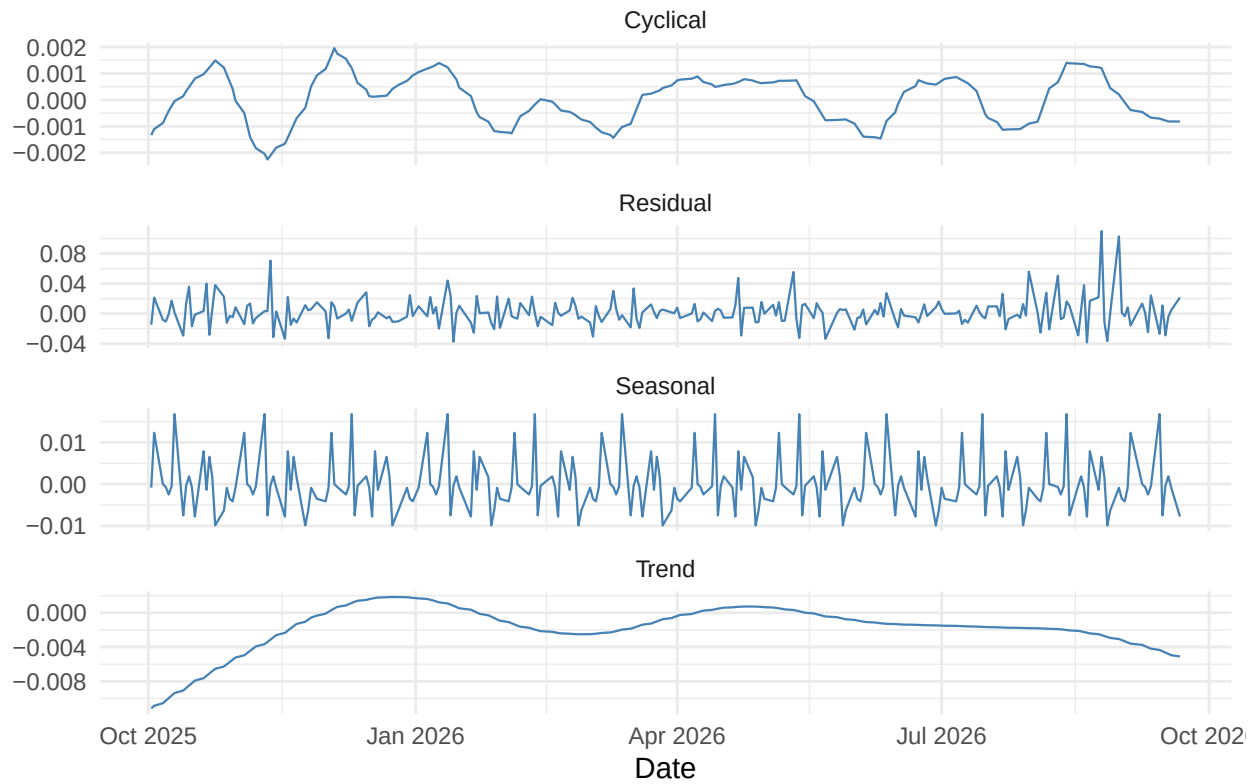
Decomposition of Daily Returns: GRAB



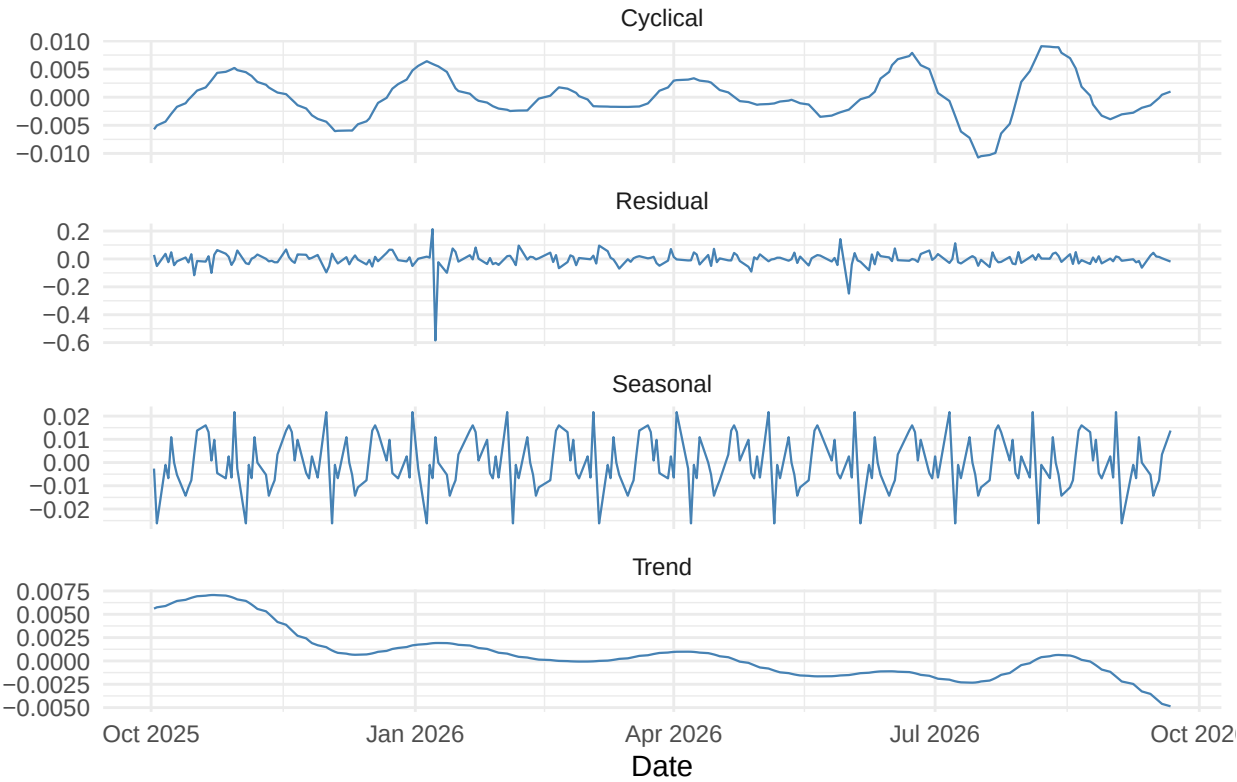
Decomposition of Daily Returns: FRMI



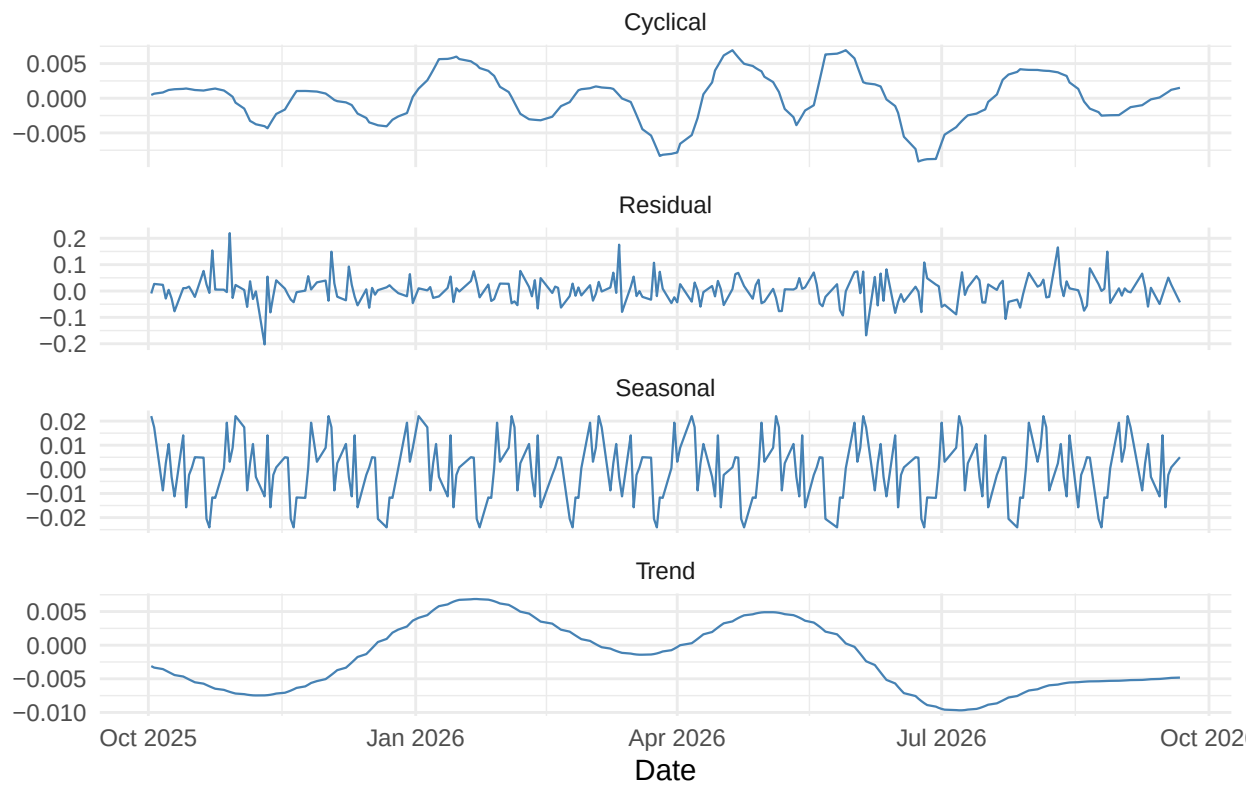
Decomposition of Daily Returns: CAAS



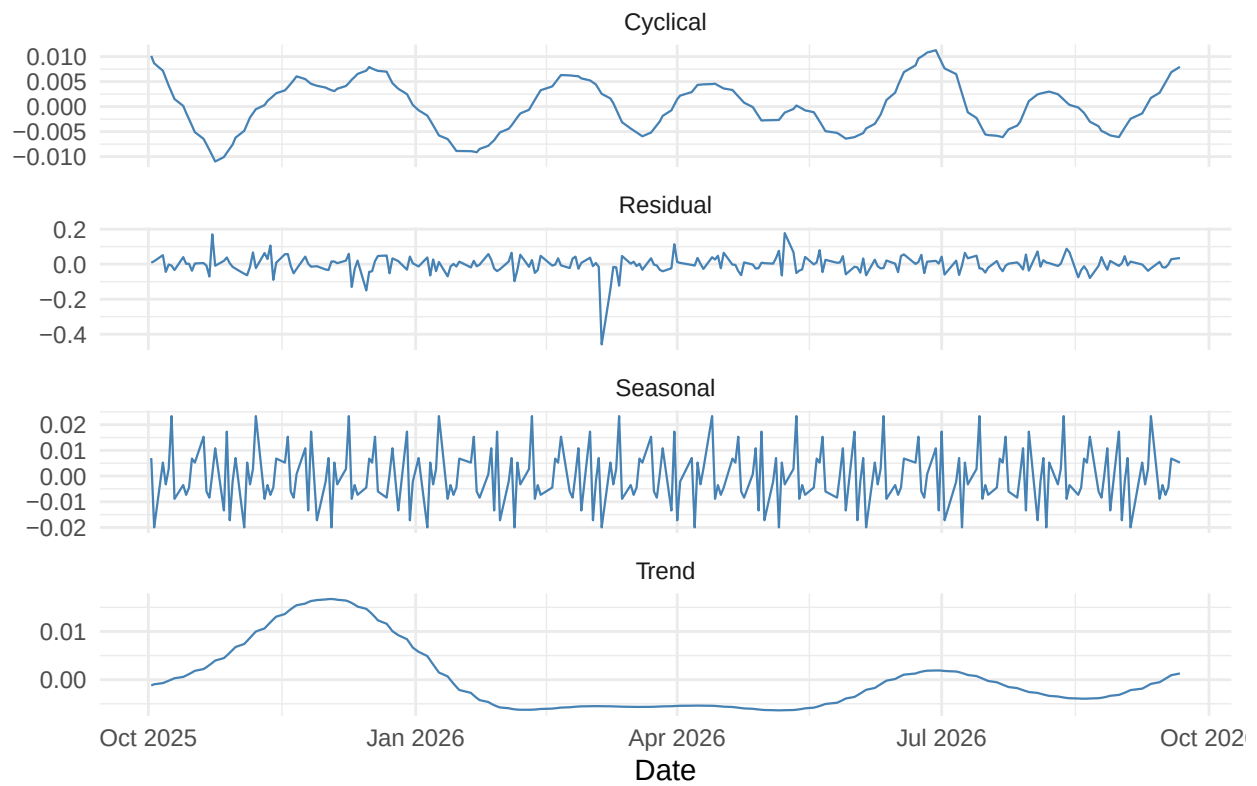
Decomposition of Daily Returns: IMRX



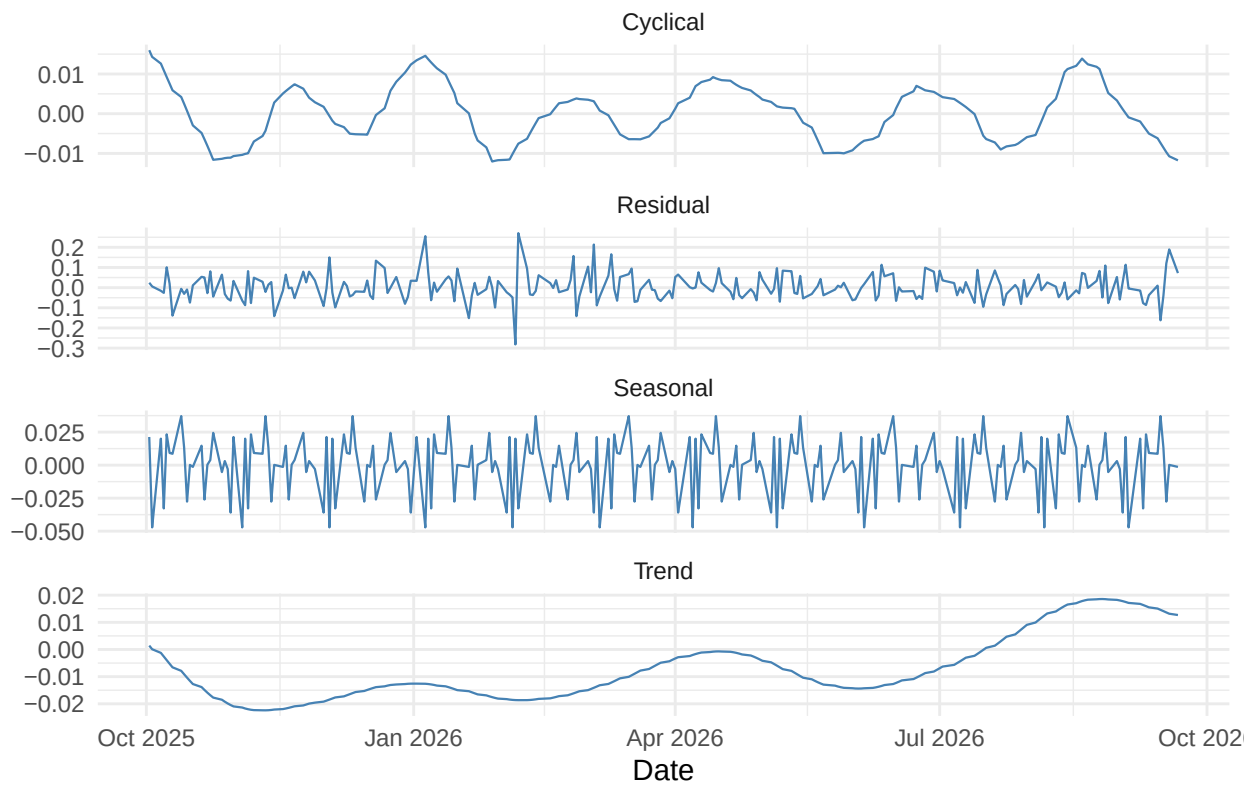
Decomposition of Daily Returns: ACDC



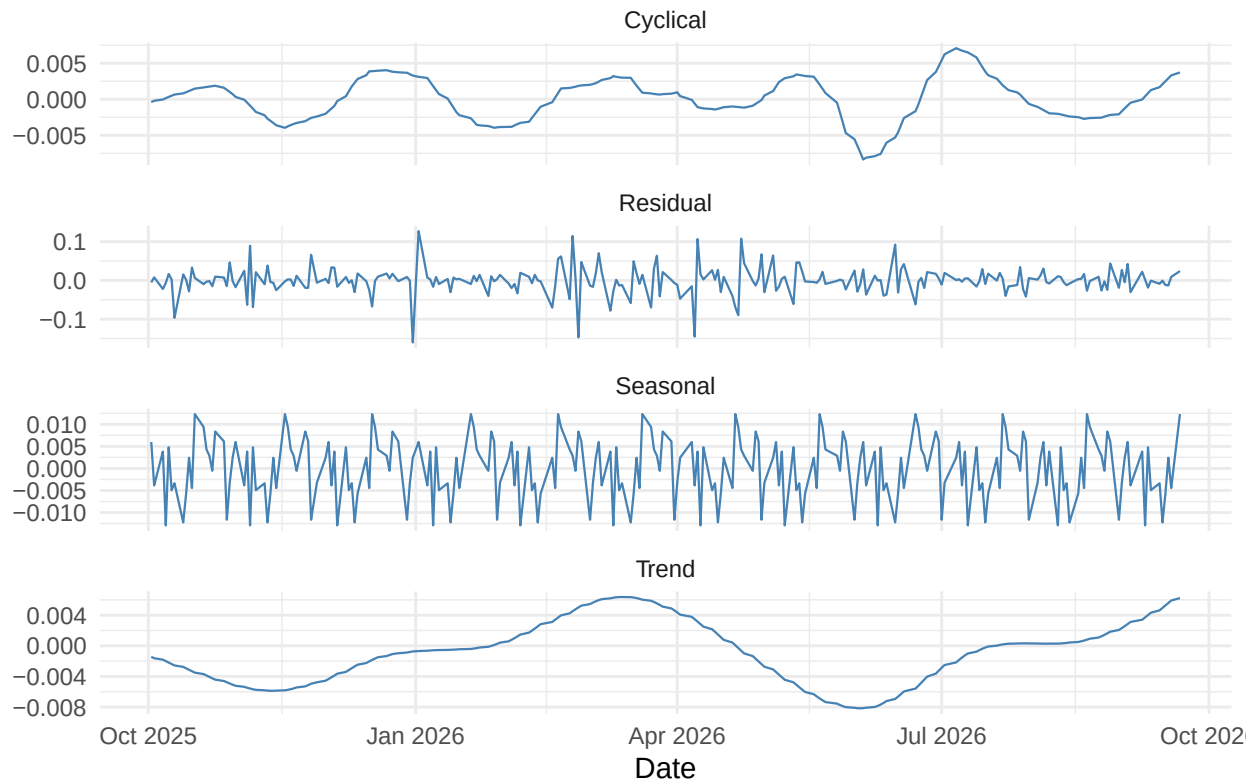
Decomposition of Daily Returns: OWLT



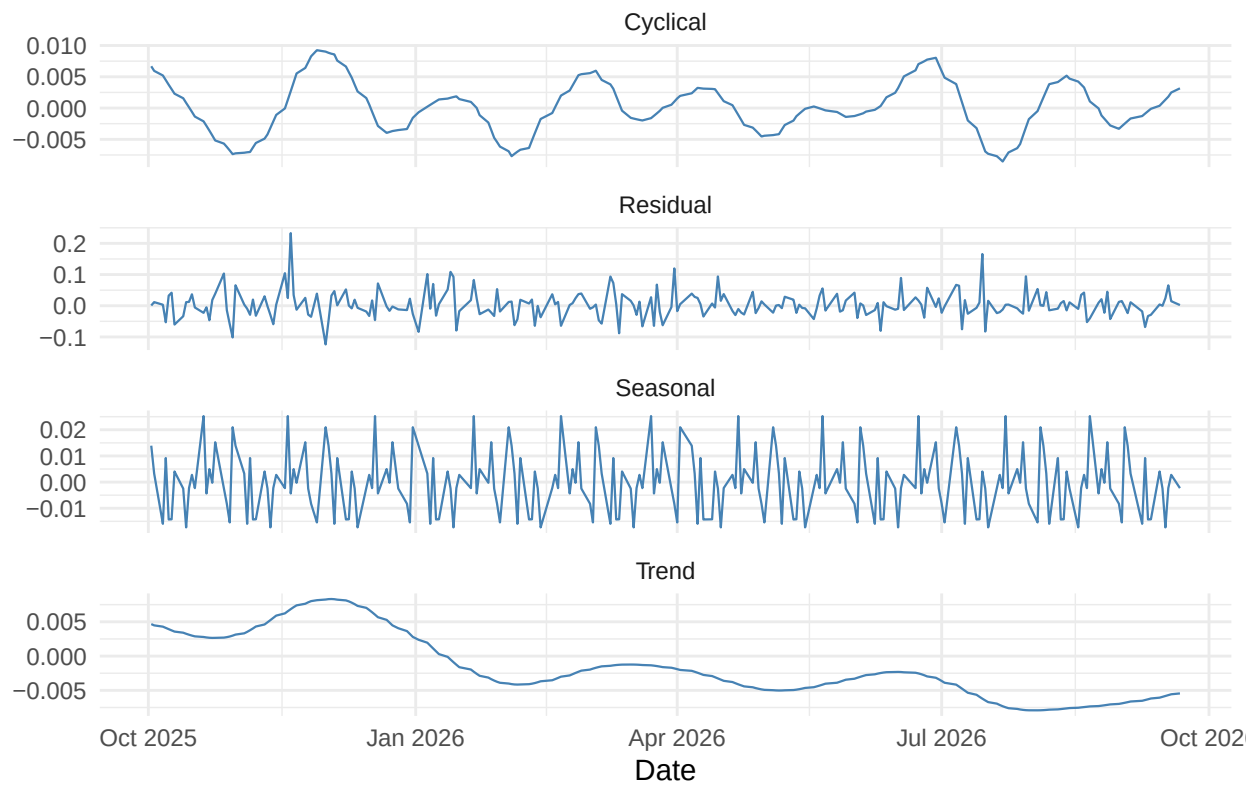
Decomposition of Daily Returns: DFDV



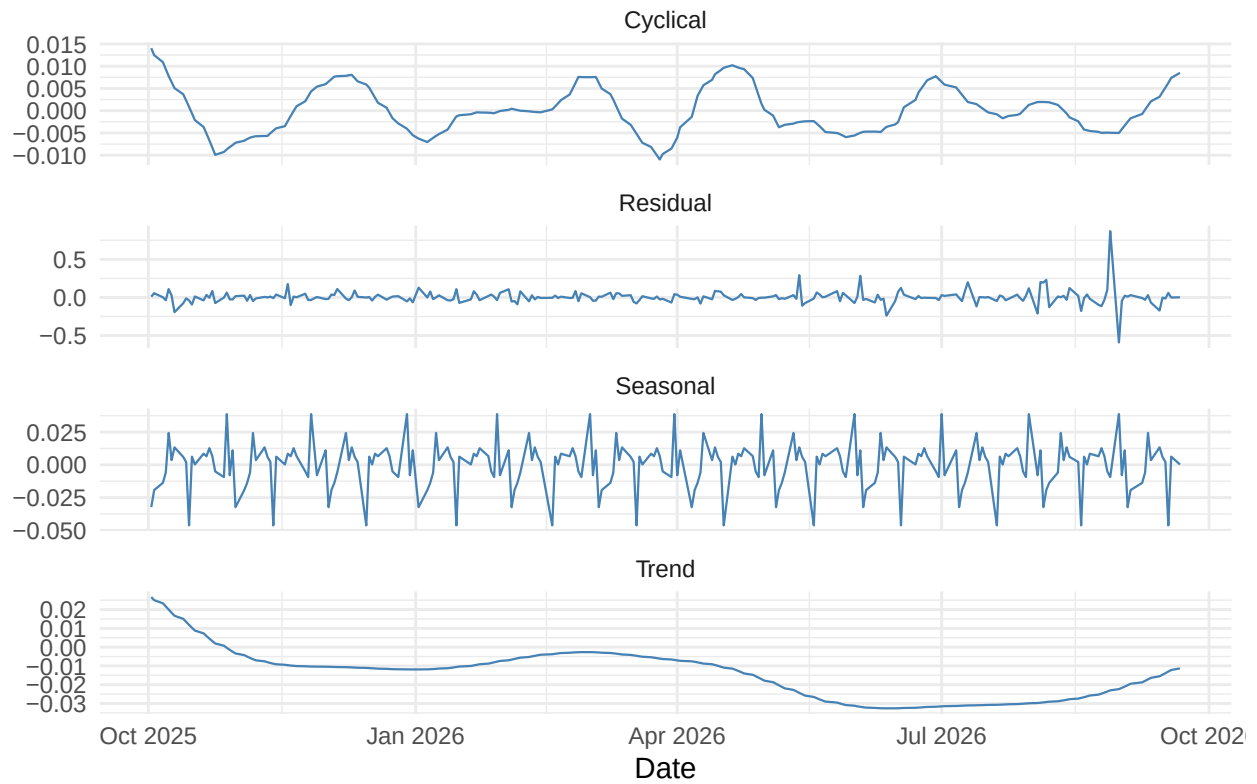
Decomposition of Daily Returns: TOUR



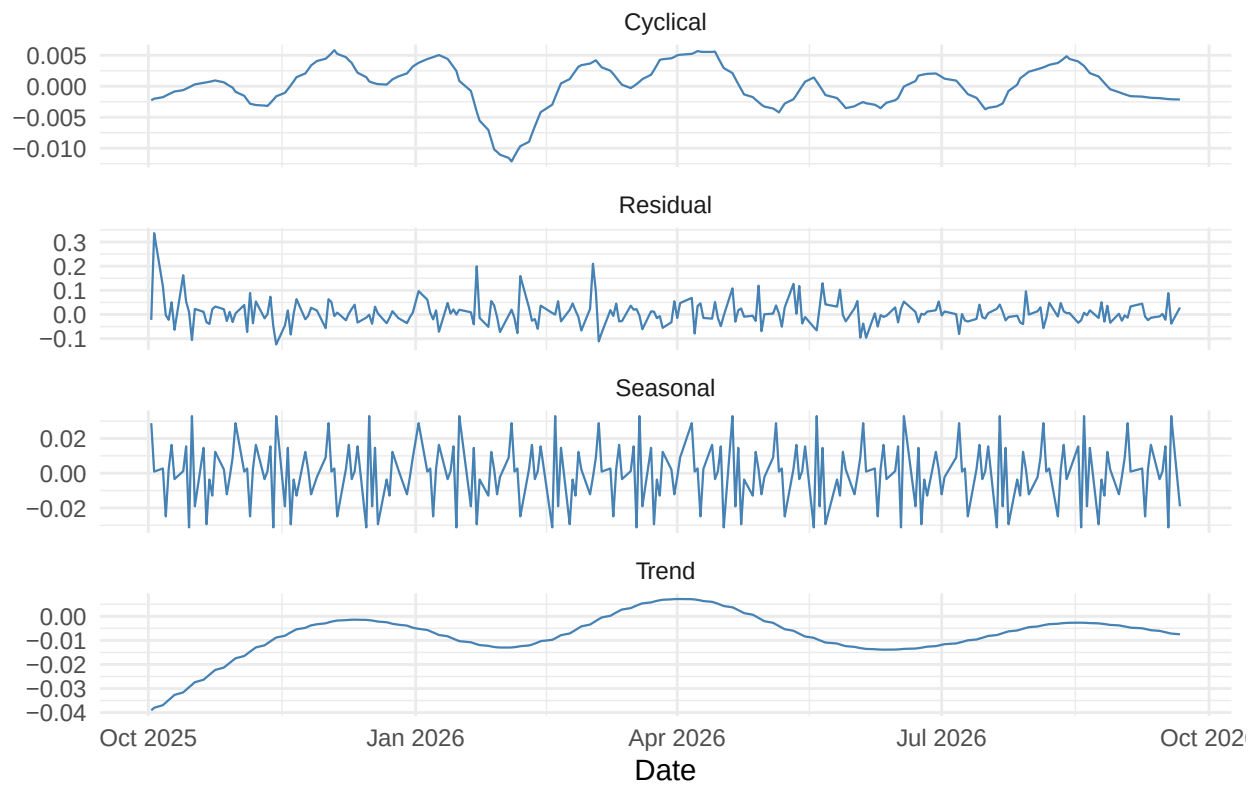
Decomposition of Daily Returns: ANN



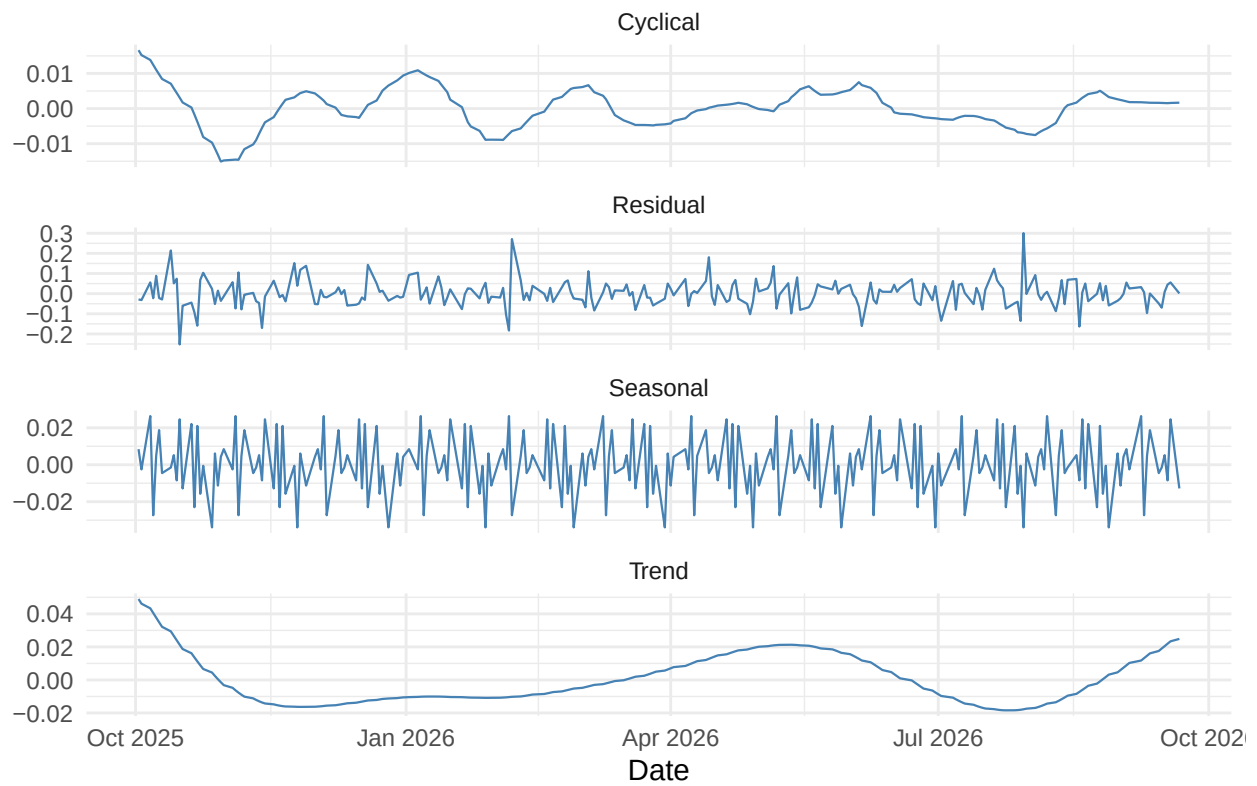
Decomposition of Daily Returns: FNGR



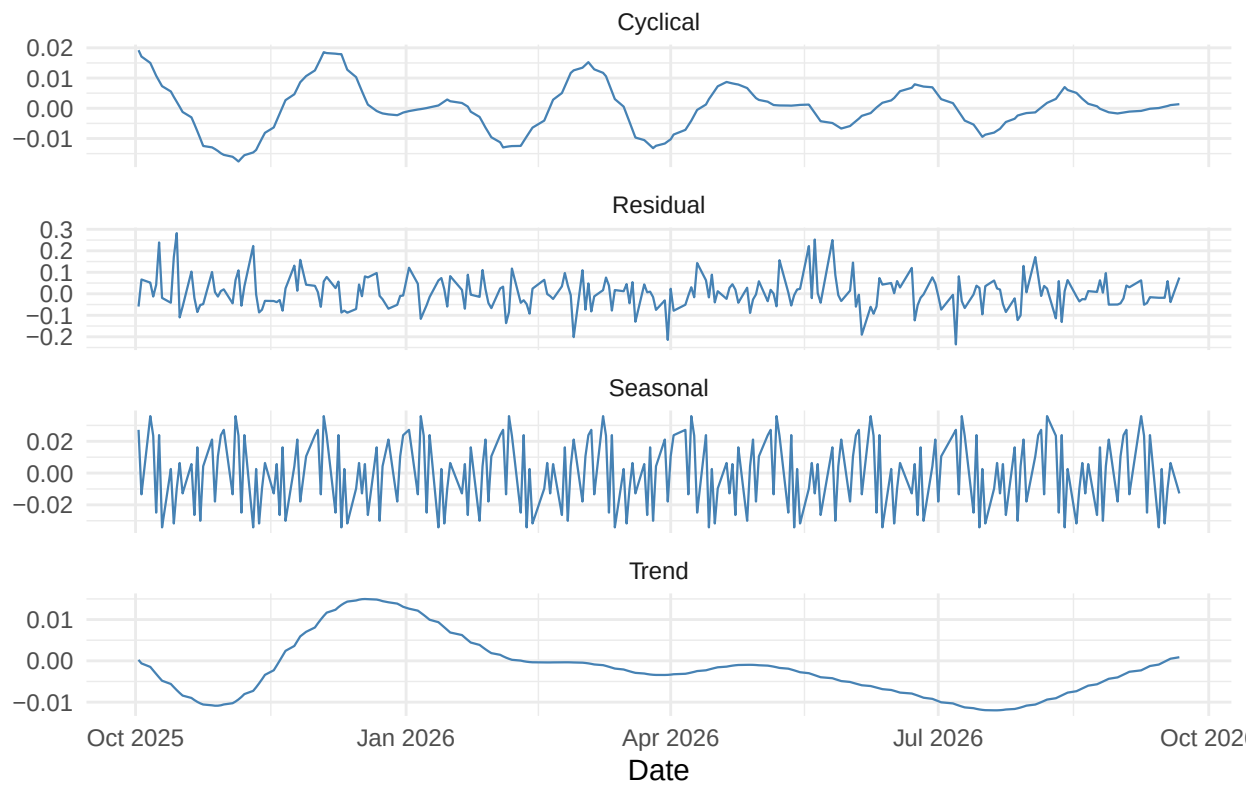
Decomposition of Daily Returns: PLUG



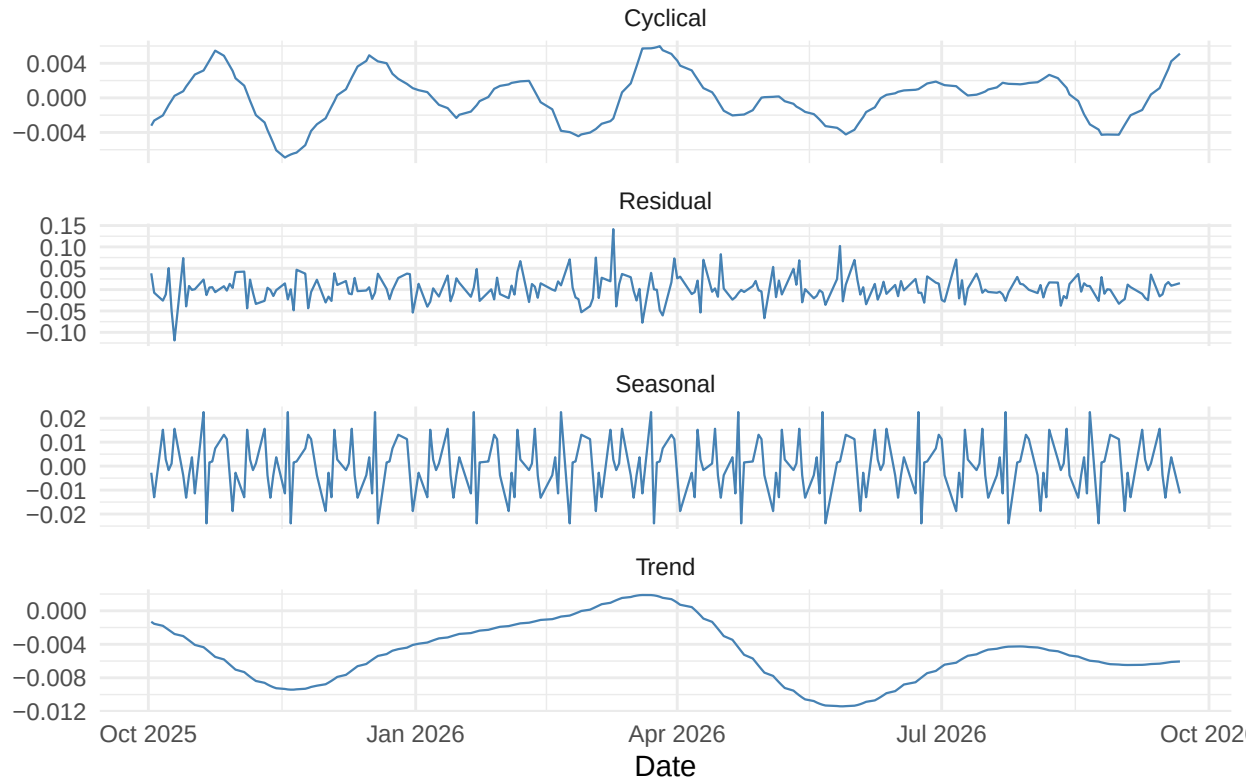
Decomposition of Daily Returns: KEEL



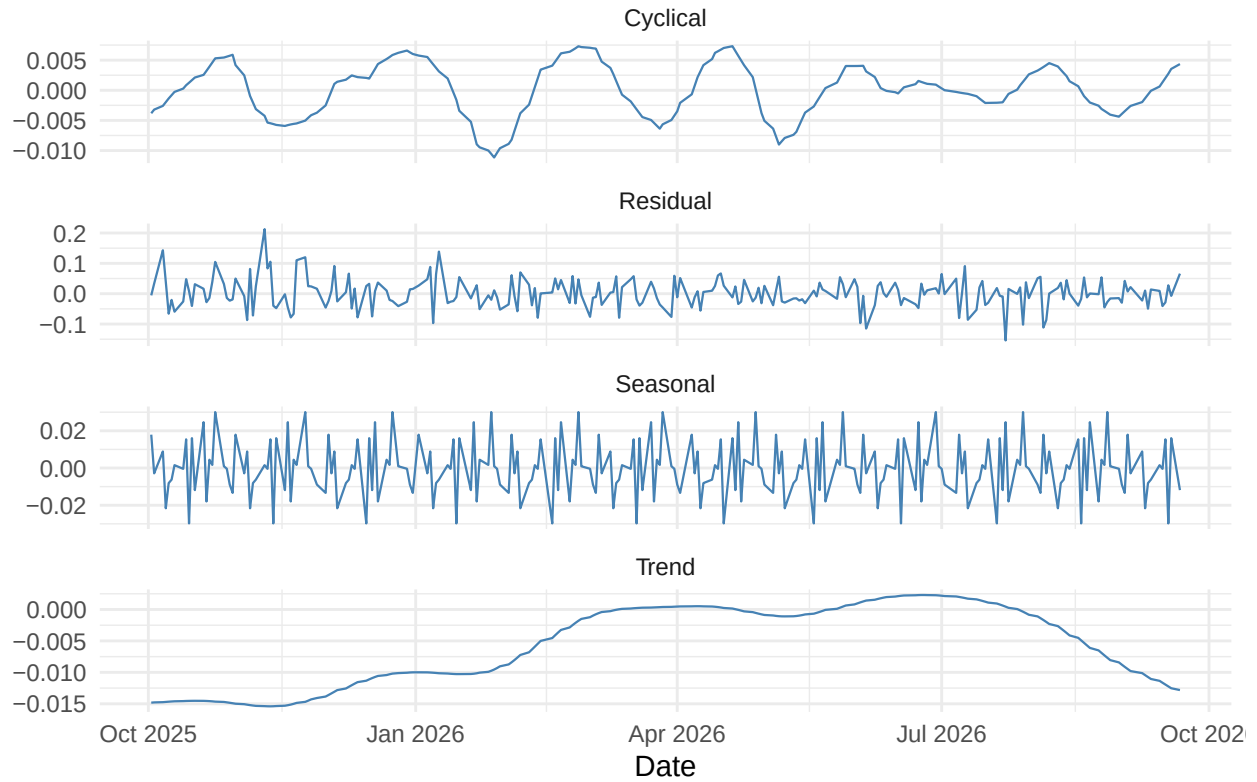
Decomposition of Daily Returns: TE



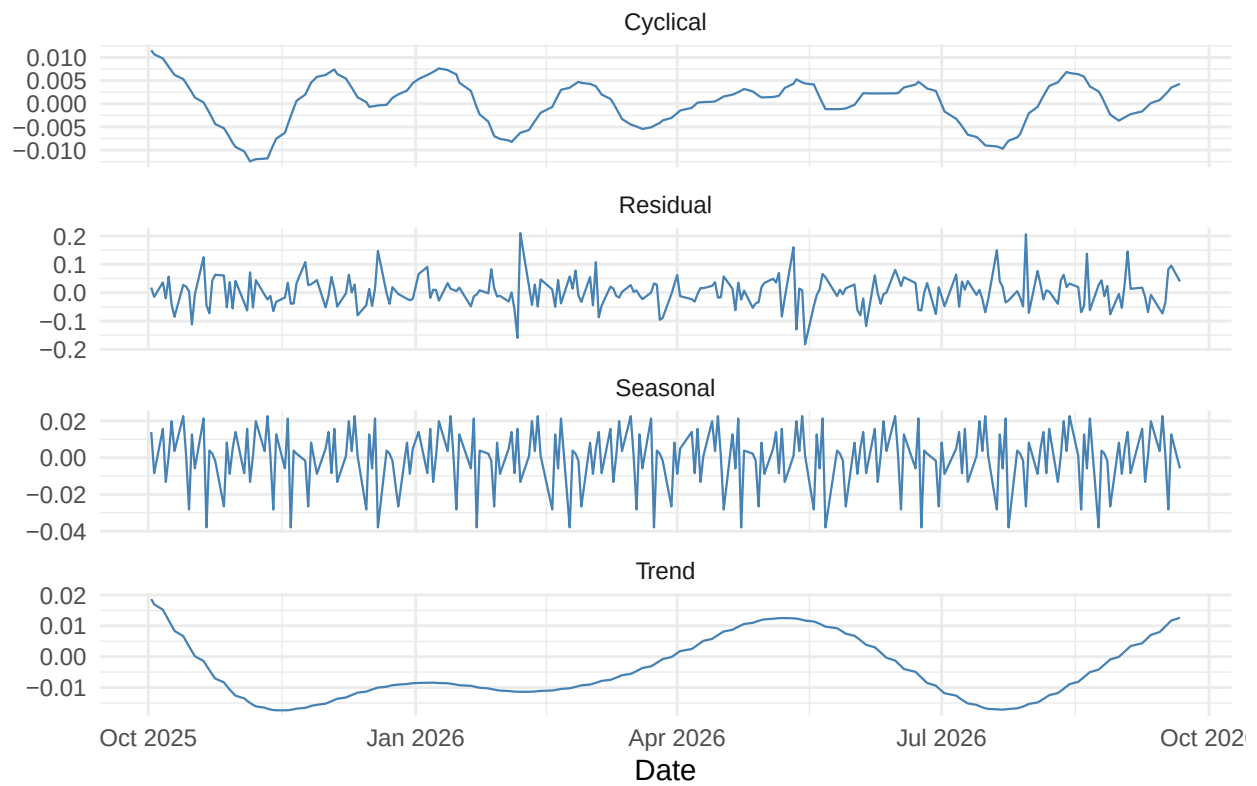
Decomposition of Daily Returns: NIO



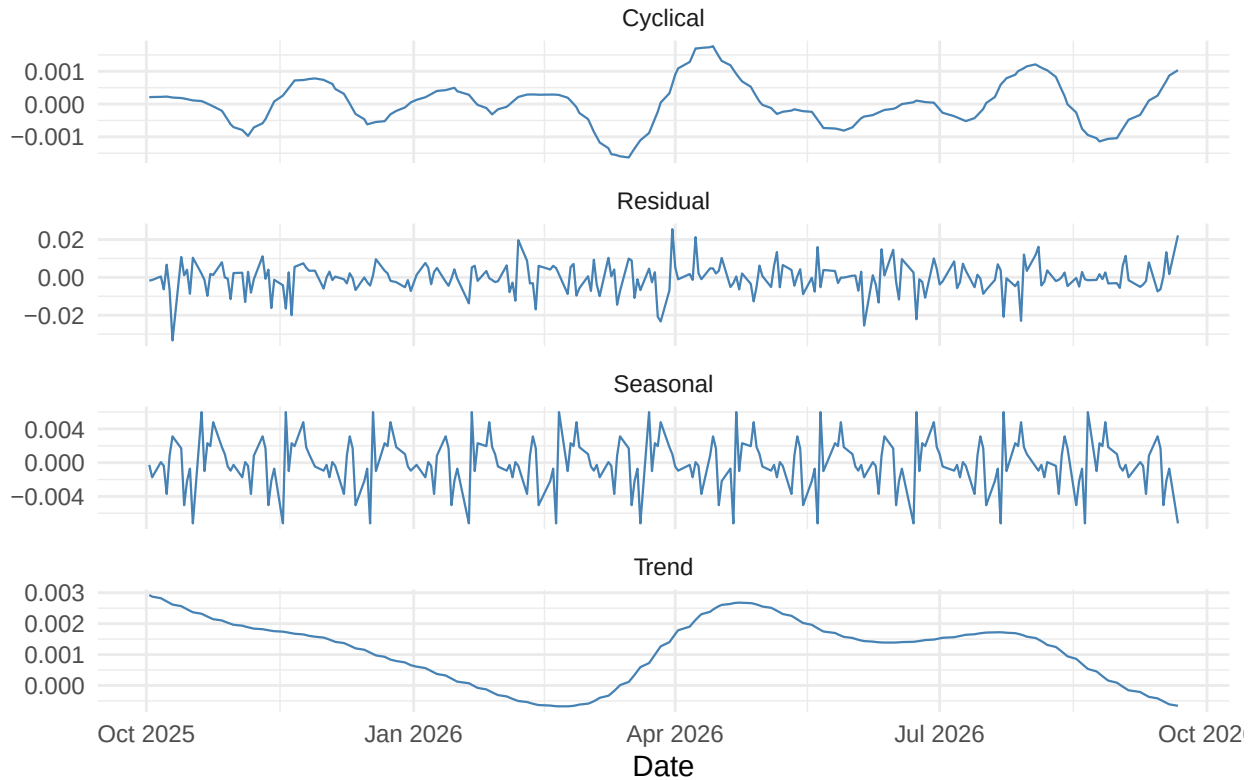
Decomposition of Daily Returns: OPEN



Decomposition of Daily Returns: BTBT



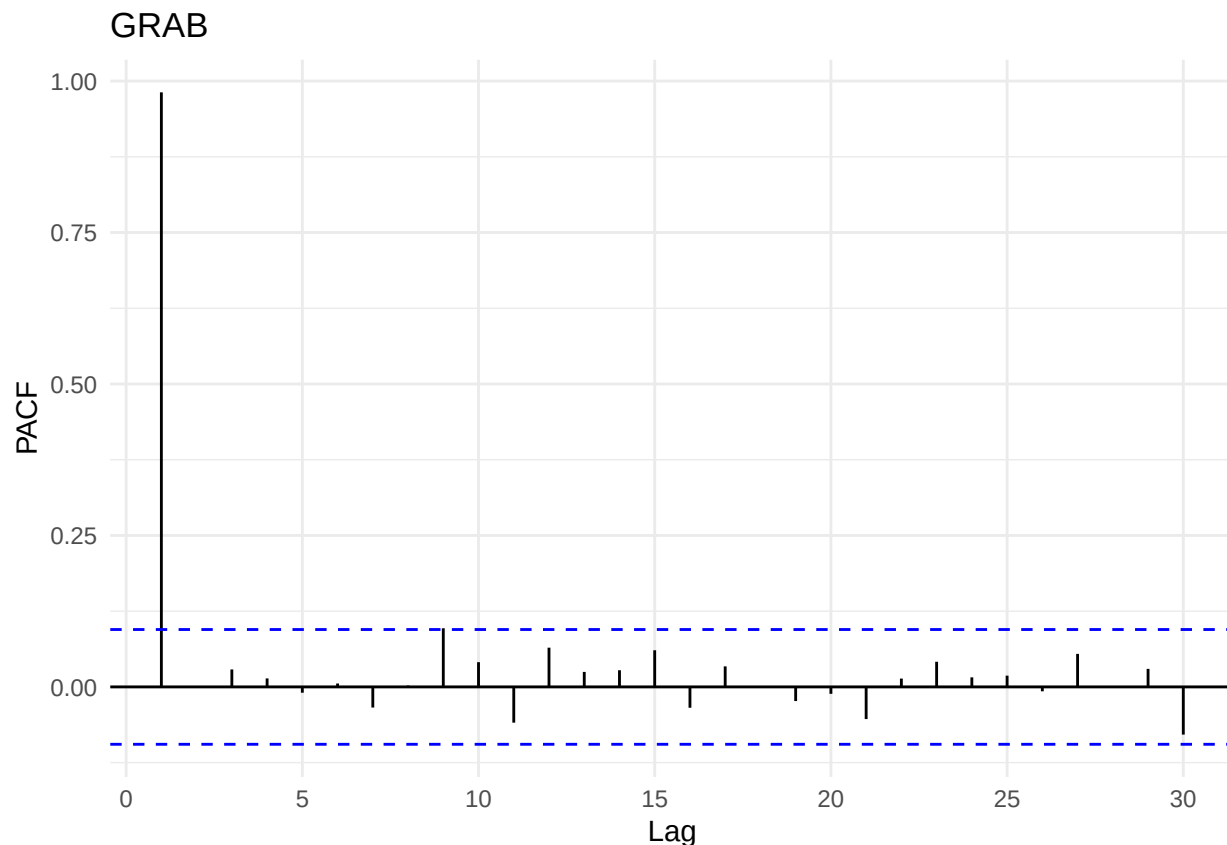
Decomposition of Daily Returns: SPY



```
## (Optional) Save plots to files
ggsave("returns_superimposed.png", width = 10, height = 6)
for (t in tickers) {
  ggsave(paste0("decomp_", t, ".png"), plot = plot_decomposition(decomp_all, t),
    width = 8, height = 8)
}

## Get ACF for all stocks in BioPharma
## Individual ACF plots using ggplot2 (nicer styling)
acf_plot_list <- lapply(colnames(prices), function(t) {
  ggPacf(as.numeric(prices[, t]), lag.max = 30) +
    ggtitle(t) +
    theme_minimal()
})

acf_plot_list[[1]]
```



```
# Test for stationarity
```

```
## ---- Loop across all stocks in `returns` (or `prices`) ----
```

```
for (t in colnames(returns_log)) {
  result <- adf.test(as.numeric(na.omit(returns_log[, t])))
  cat(sprintf("%-6s ADF stat: %.4f p-value: %.4f %s\n",
    t, result$statistic, result$p.value,
    ifelse(result$p.value < 0.05, "-> Stationary", "-> Non-stationary")))
}
```

```
## GRAB ADF stat: -5.7021 p-value: 0.0100 -> Stationary
## FRMI ADF stat: -6.7644 p-value: 0.0100 -> Stationary
## CAAS ADF stat: -5.6420 p-value: 0.0100 -> Stationary
## IMRX ADF stat: -6.4372 p-value: 0.0100 -> Stationary
## ACDC ADF stat: -5.7360 p-value: 0.0100 -> Stationary
## OWLT ADF stat: -5.8008 p-value: 0.0100 -> Stationary
## DFDV ADF stat: -5.9256 p-value: 0.0100 -> Stationary
## TOUR ADF stat: -7.0367 p-value: 0.0100 -> Stationary
## ANNX ADF stat: -5.9155 p-value: 0.0100 -> Stationary
## FNGR ADF stat: -7.1173 p-value: 0.0100 -> Stationary
## PLUG ADF stat: -5.2907 p-value: 0.0100 -> Stationary
## KEEL ADF stat: -6.1314 p-value: 0.0100 -> Stationary
## TE ADF stat: -5.8907 p-value: 0.0100 -> Stationary
## NIO ADF stat: -6.6579 p-value: 0.0100 -> Stationary
## OPEN ADF stat: -7.9016 p-value: 0.0100 -> Stationary
## BTBT ADF stat: -6.7346 p-value: 0.0100 -> Stationary
## SPY ADF stat: -6.2106 p-value: 0.0100 -> Stationary
```

Alpha and Beta for each stock

```
#head(returns_log)
## Risk free interest rate daily
rf_daily <- 0.0001598

# SPY excess return
returns_log$SPY_excess <- returns_log$SPY - rf_daily

# Make sure SPY is available
if (!"SPY_excess" %in% colnames(returns_log)) {
  stop("SPY was not successfully downloaded.")
}

# All stocks except SPY_excess
stock_tickers <- setdiff(colnames(returns_log), "SPY_excess")

# Store regression models
models <- list()

for(stock in stock_tickers) {

  # Create temporary data frame
  reg_data <- data.frame(
    Stock_Return = as.numeric(returns_log[, stock] - rf_daily),
    SPY_excess_Return = as.numeric(returns_log[, "SPY_excess"])
  )

  # Remove missing observations
  reg_data <- na.omit(reg_data)

  # Regression
  models[[stock]] <- lm(
    Stock_Return ~ SPY_excess_Return,
    data = reg_data
  )
}

# Creating a summary table for each stock
regression_results <- do.call(
  rbind,
  lapply(stock_tickers, function(stock) {

    model <- models[[stock]]
    sm <- summary(model)

    data.frame(
      Ticker = stock,

      Alpha = coef(model)[1],
      Beta = coef(model)[2],

      Alpha_SE = sm$coefficients[1, 2],
      Beta_SE = sm$coefficients[2, 2],
```

```

Alpha_pvalue = sm$coefficients[1, 4],
Beta_pvalue = sm$coefficients[2, 4],

R_squared = sm$r.squared,
Adjusted_R_squared = sm$adj.r.squared,

N = nobs(model)
)
})
)

rownames(regression_results) <- NULL

regression_results

##      Ticker      Alpha      Beta      Alpha_SE      Beta_SE Alpha_pvalue
## 1   GRAB -3.799201e-03 1.5066175 1.299770e-03 1.570486e-01 0.003796858
## 2   FRMI -9.104913e-03 2.6544810 4.795147e-03 5.793878e-01 0.058788287
## 3   CAAS 2.254498e-05 0.3017799 1.342183e-03 1.621732e-01 0.986612261
## 4   IMRX -2.199291e-03 1.1557647 3.610764e-03 4.362813e-01 0.543036505
## 5   ACDC 4.706133e-04 0.6106328 3.420792e-03 4.133274e-01 0.890691697
## 6   OWLT -3.428131e-03 2.3330425 3.323836e-03 4.016123e-01 0.303397587
## 7   DFDV -5.586070e-03 3.9762626 4.066600e-03 4.913590e-01 0.170827941
## 8   TOUR -2.231419e-03 0.8481494 2.295306e-03 2.773372e-01 0.331942589
## 9   ANNX 4.443546e-04 1.5650562 2.810905e-03 3.396359e-01 0.874524229
## 10  FNGR -1.067392e-02 1.4527012 6.095675e-03 7.365280e-01 0.081207085
## 11  PLUG -2.684410e-03 2.3386543 3.320459e-03 4.012043e-01 0.419631466
## 12  KEEL -6.475290e-04 4.1789192 3.906599e-03 4.720263e-01 0.868490592
## 13   TE 6.338431e-04 3.6377300 4.865509e-03 5.878895e-01 0.896459326
## 14  NIO -3.846944e-03 1.3699085 1.970316e-03 2.380693e-01 0.052042573
## 15  OPEN -6.150807e-03 3.0095662 2.837822e-03 3.428883e-01 0.031181100
## 16  BTBT -4.356718e-03 3.6683965 3.102698e-03 3.748926e-01 0.161555278
## 17  SPY 0.000000e+00 1.0000000 3.970888e-20 4.797943e-18 1.000000000
##      Beta_pvalue      R_squared Adjusted_R_squared      N
## 1 1.144756e-18 0.276345477 0.273342761 243
## 2 7.408691e-06 0.080118928 0.076301994 243
## 3 6.398329e-02 0.014164774 0.010074172 243
## 4 8.603514e-03 0.028295845 0.024263877 243
## 5 1.408851e-01 0.008975103 0.004862967 243
## 6 1.975044e-08 0.122828388 0.119188672 243
## 7 2.882290e-14 0.213668643 0.210405858 243
## 8 2.478158e-03 0.037357390 0.033363022 243
## 9 6.591481e-06 0.080973649 0.077160262 243
## 10 4.971137e-02 0.015885564 0.011802101 243
## 11 1.779364e-08 0.123567034 0.119930382 243
## 12 1.881334e-16 0.245408820 0.242277736 243
## 13 2.600220e-09 0.137093307 0.133512782 243
## 14 2.631176e-08 0.120795149 0.117146996 243
## 15 3.143050e-16 0.242227860 0.239083578 243
## 16 2.962556e-19 0.284335529 0.281365967 243
## 17 0.000000e+00 1.000000000 1.000000000 243

# Regression plot for each stock and SPY excess
library(ggplot2)

```

```

regression_plots <- list()

for(stock in stock_tickers) {

  plot_data <- data.frame(
    Stock_Return = as.numeric(returns_log[, stock]),
    SPY_Return   = as.numeric(returns_log[, "SPY"])
  )

  plot_data <- na.omit(plot_data)

  p <- ggplot(
    plot_data,
    aes(x = SPY_Return, y = Stock_Return)
  ) +
    geom_point(alpha = 0.6) +
    geom_smooth(
      method = "lm",
      se = TRUE
    ) +
    labs(
      title = paste(stock, "Returns vs SPY Returns"),
      x = "SPY Daily Log Return",
      y = paste(stock, "Daily Log Return")
    ) +
    theme_minimal()

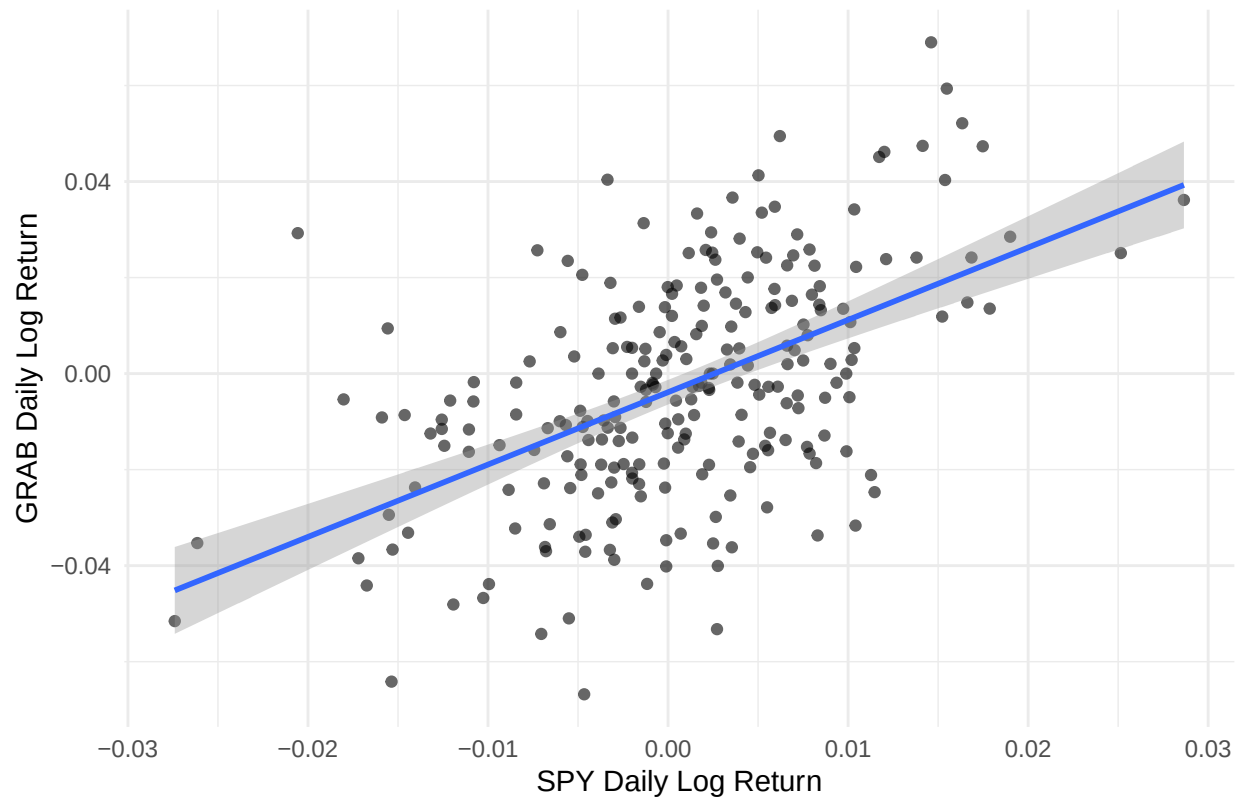
  regression_plots[[stock]] <- p
}

regression_plots

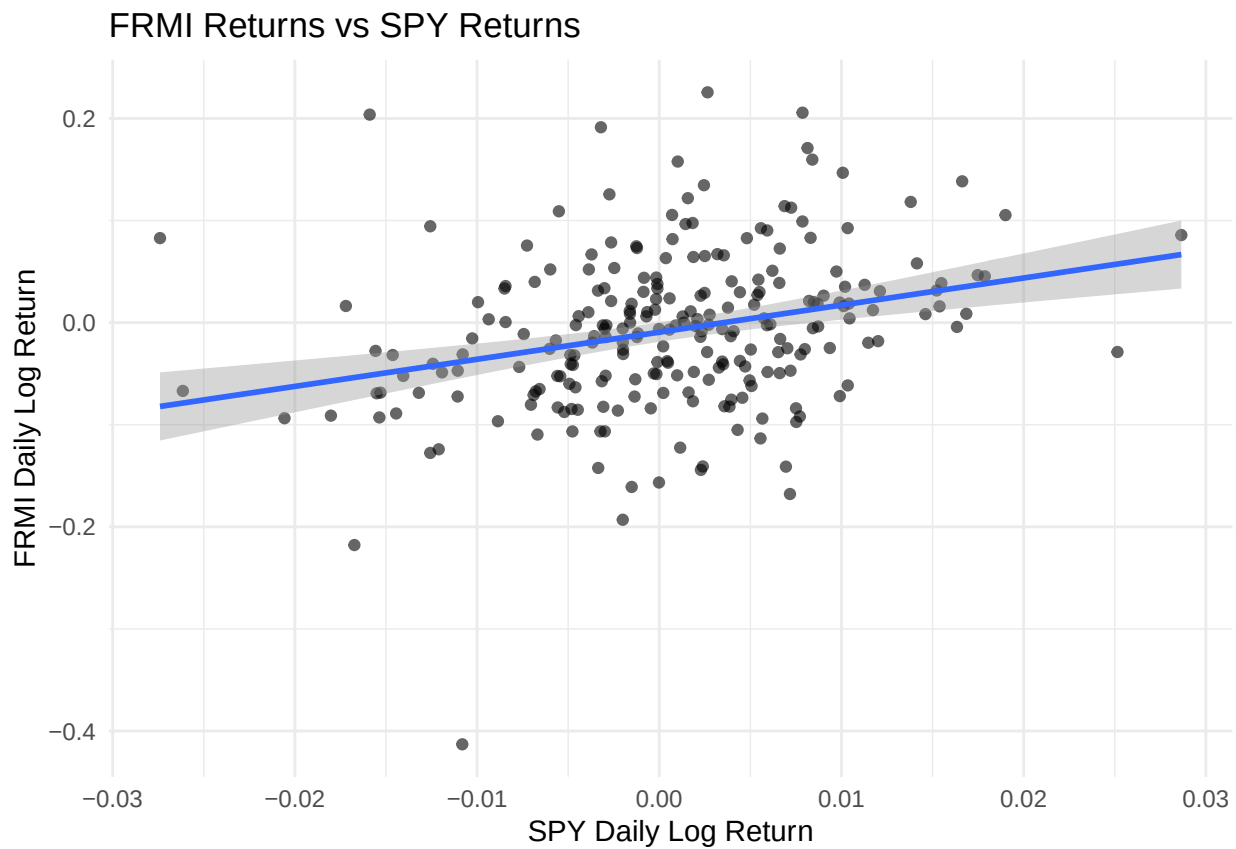
```

```
## $GRAB
```

GRAB Returns vs SPY Returns

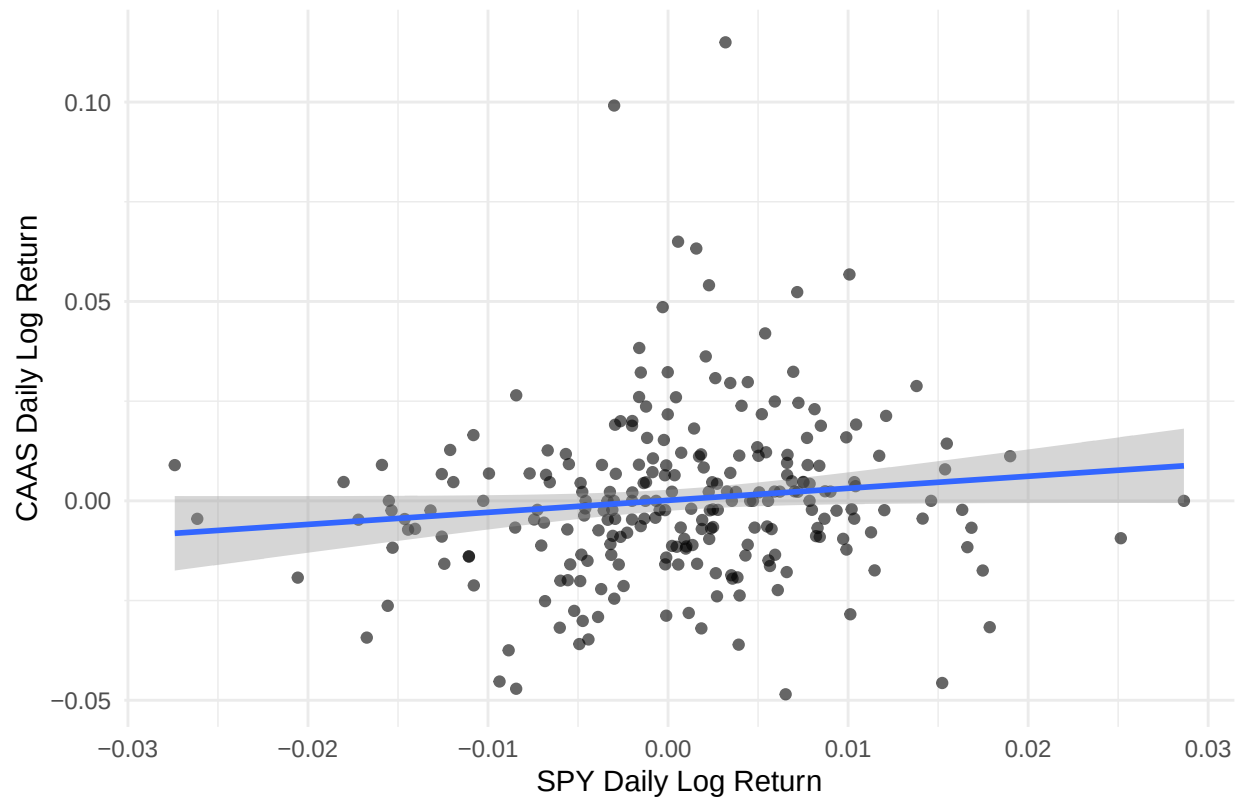


\$FRMI

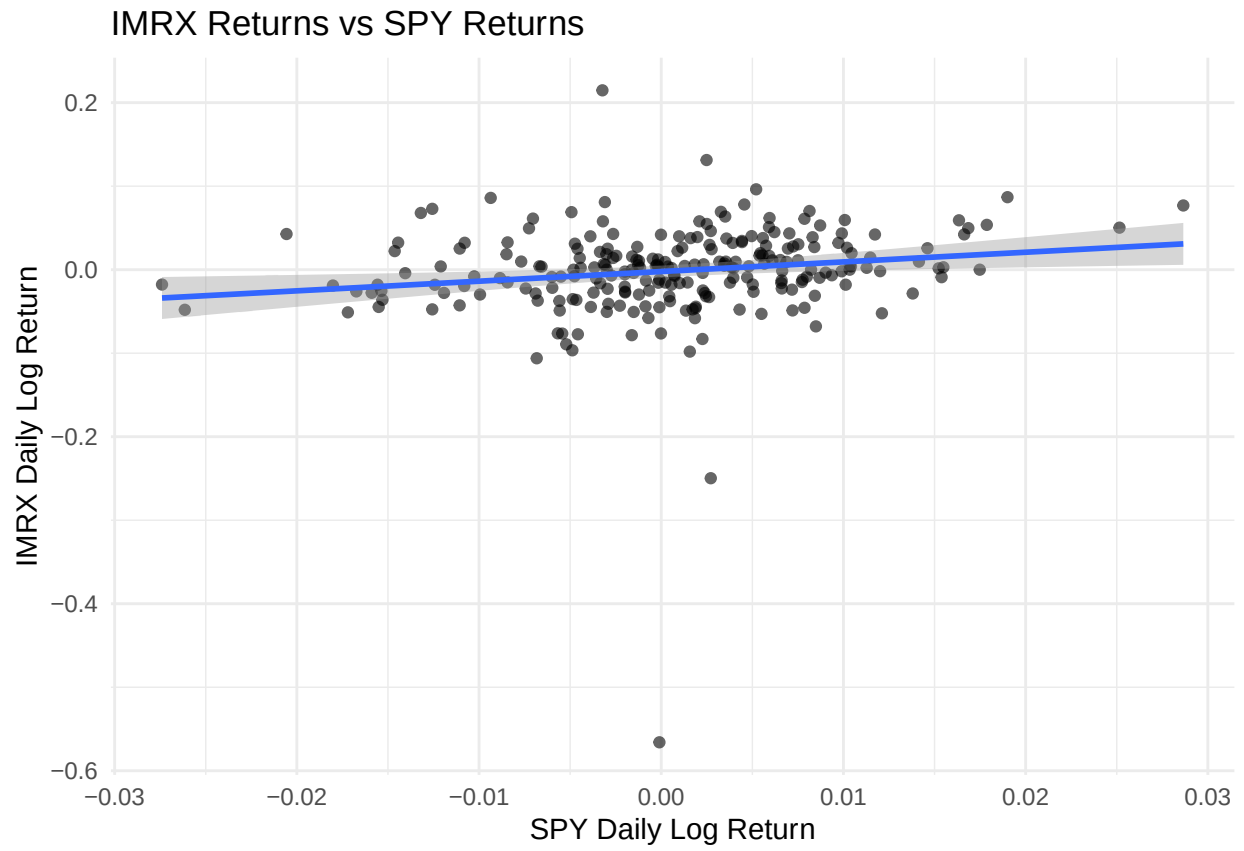


\$CAAS

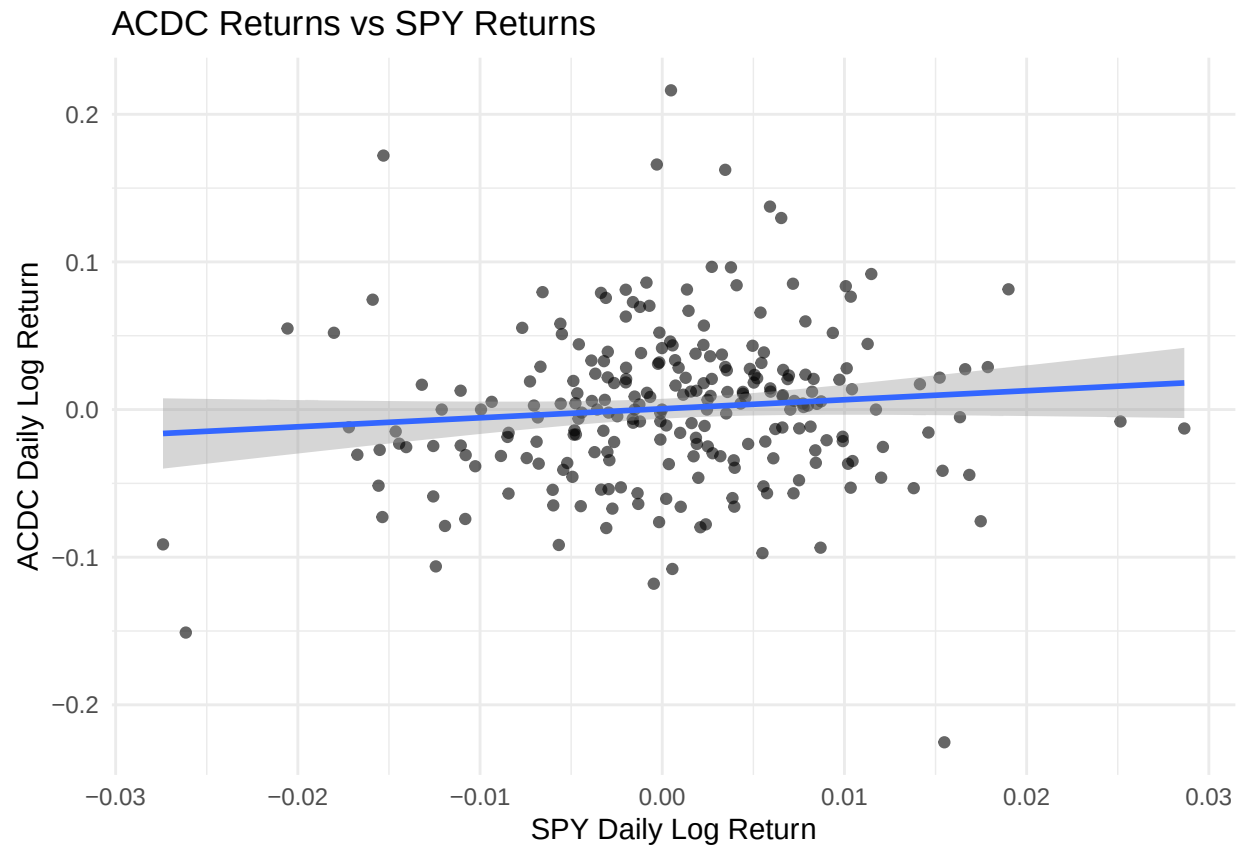
CAAS Returns vs SPY Returns



\$IMRX

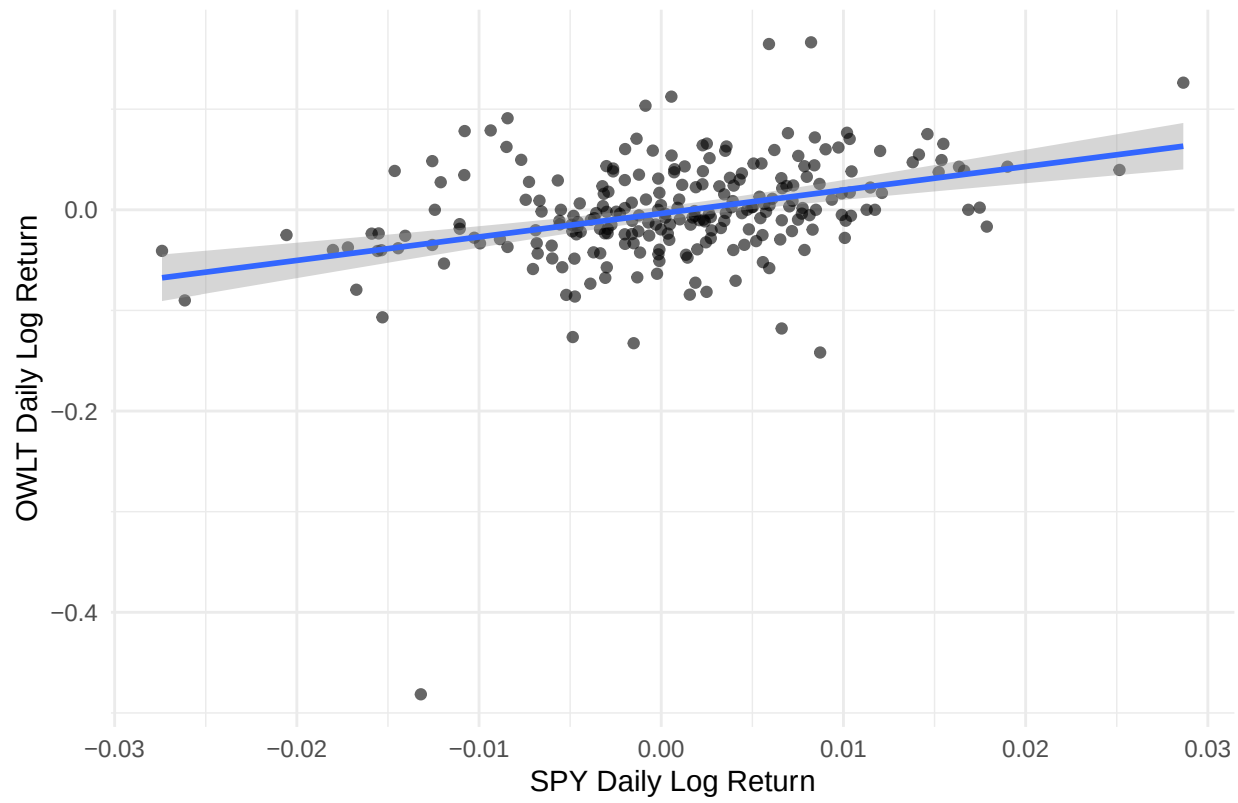


\$ACDC

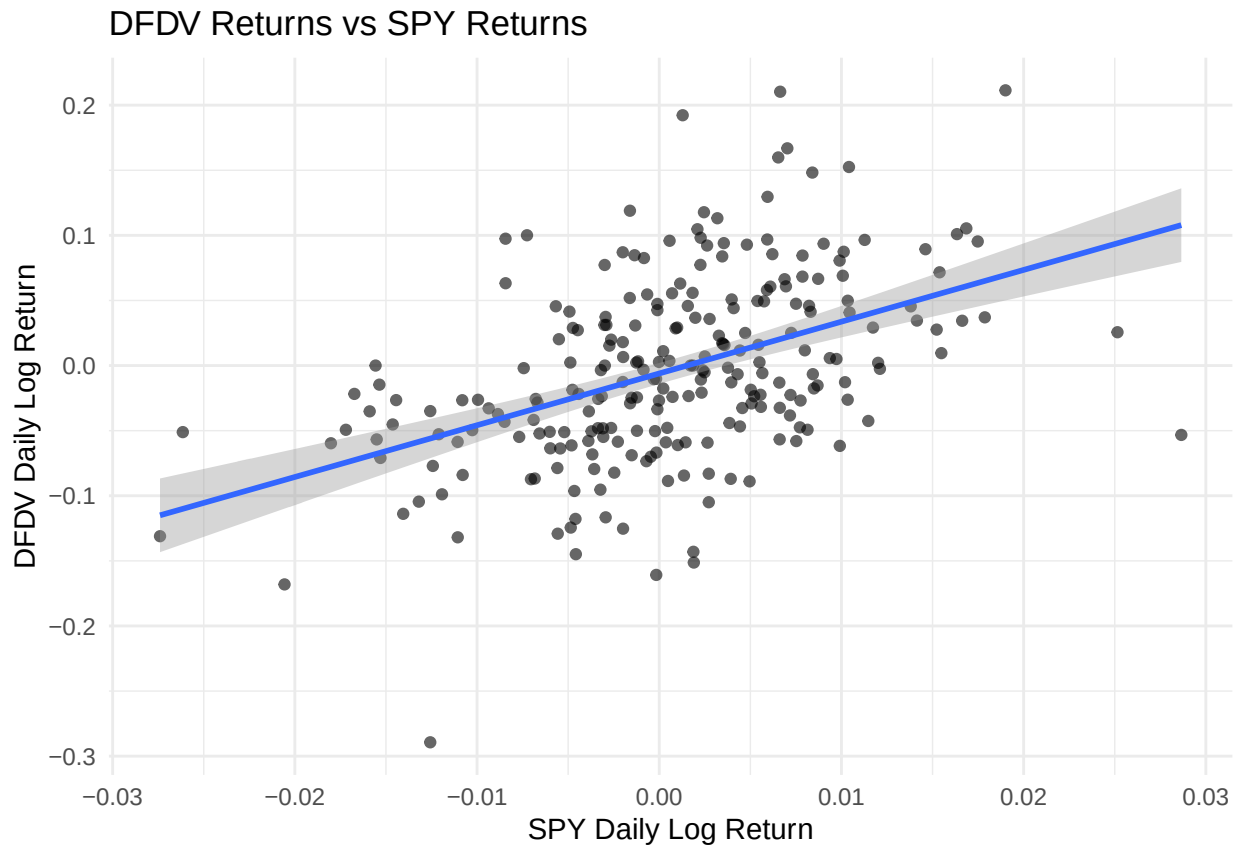


\$OWLT

OWLT Returns vs SPY Returns

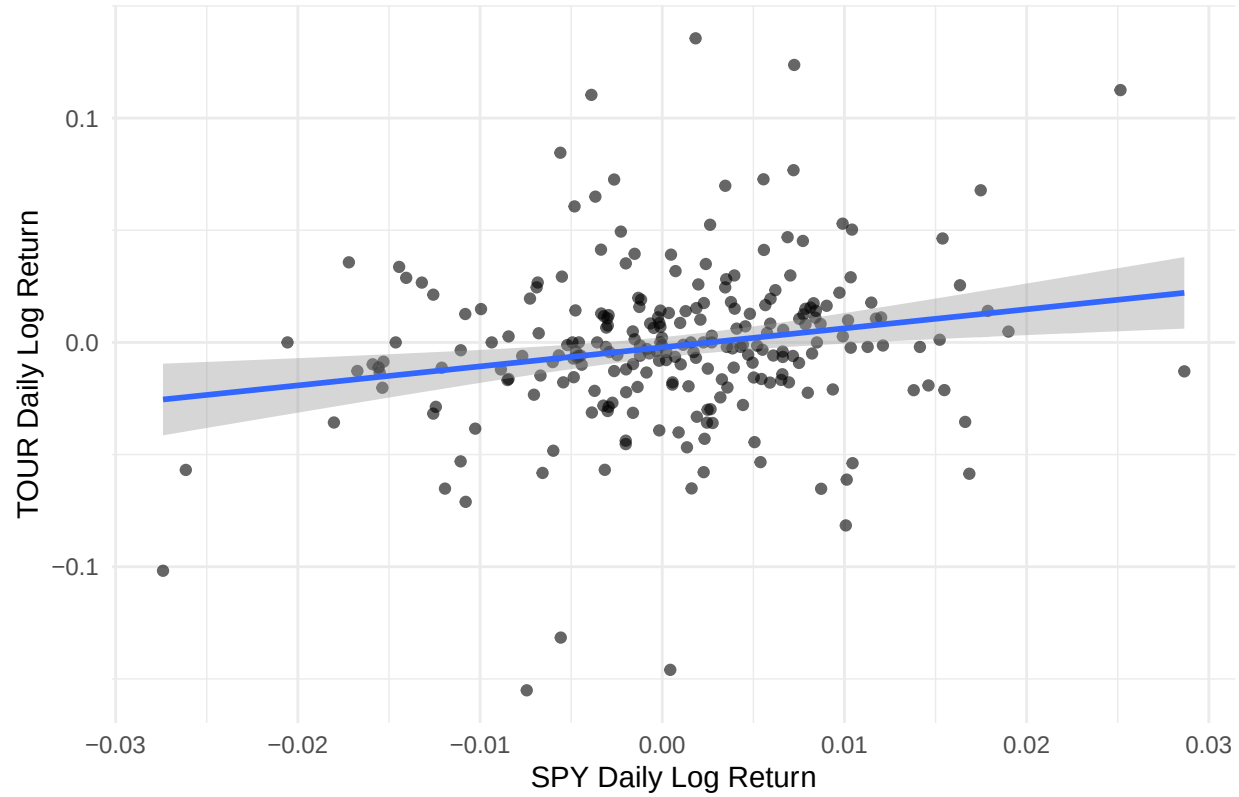


\$DFDV



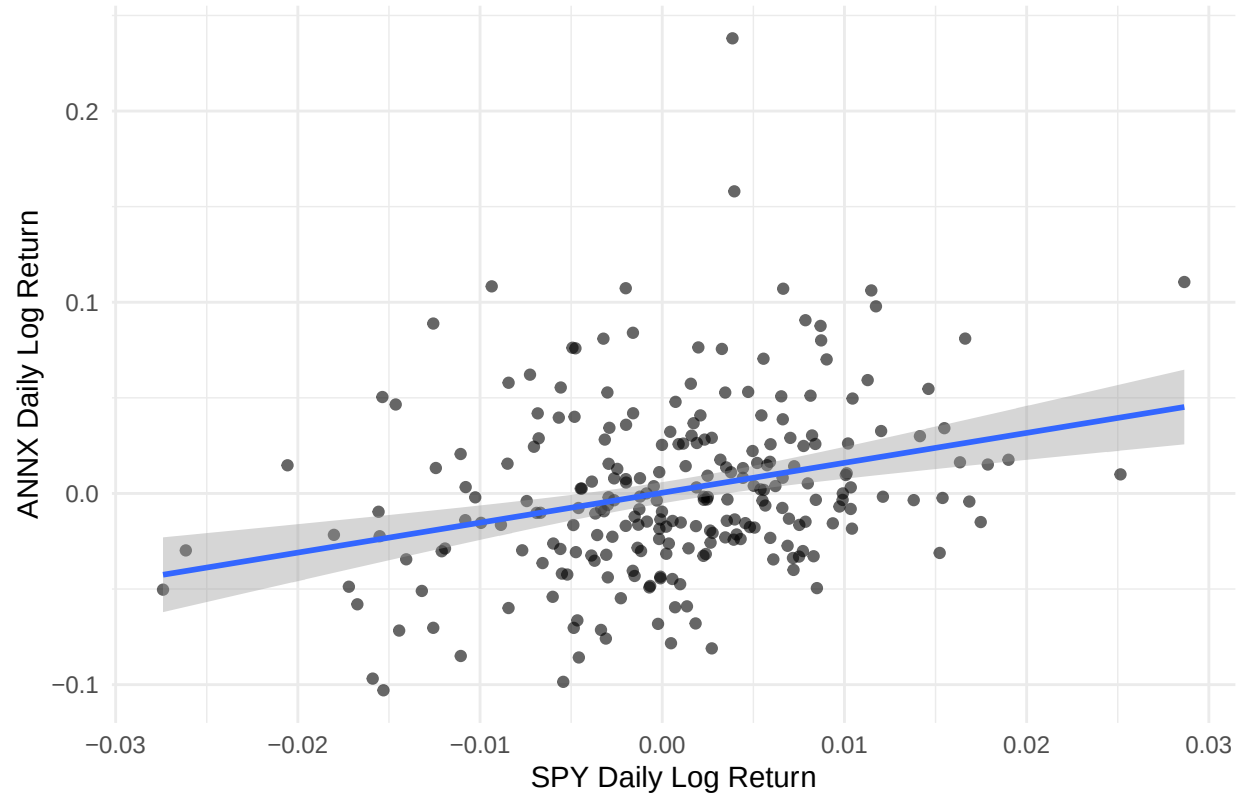
\$TOUR

TOUR Returns vs SPY Returns



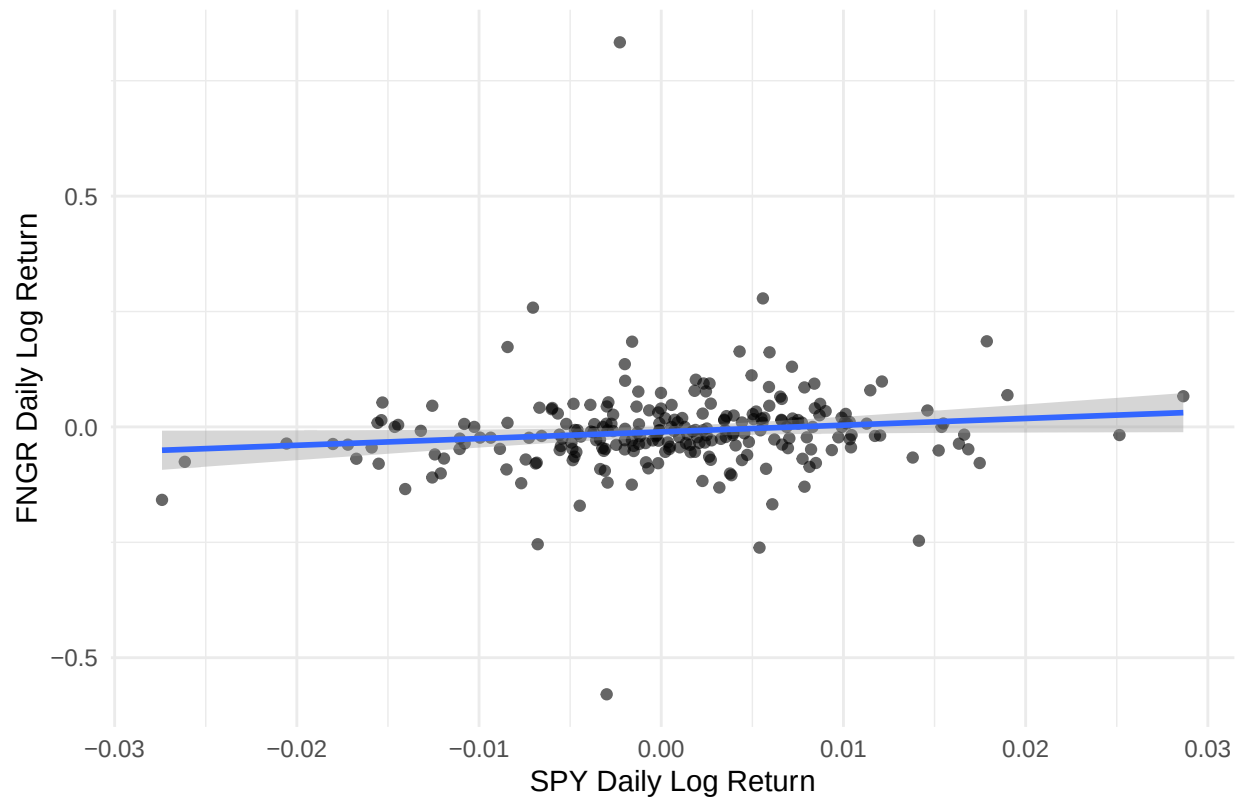
\$ANX

ANNX Returns vs SPY Returns

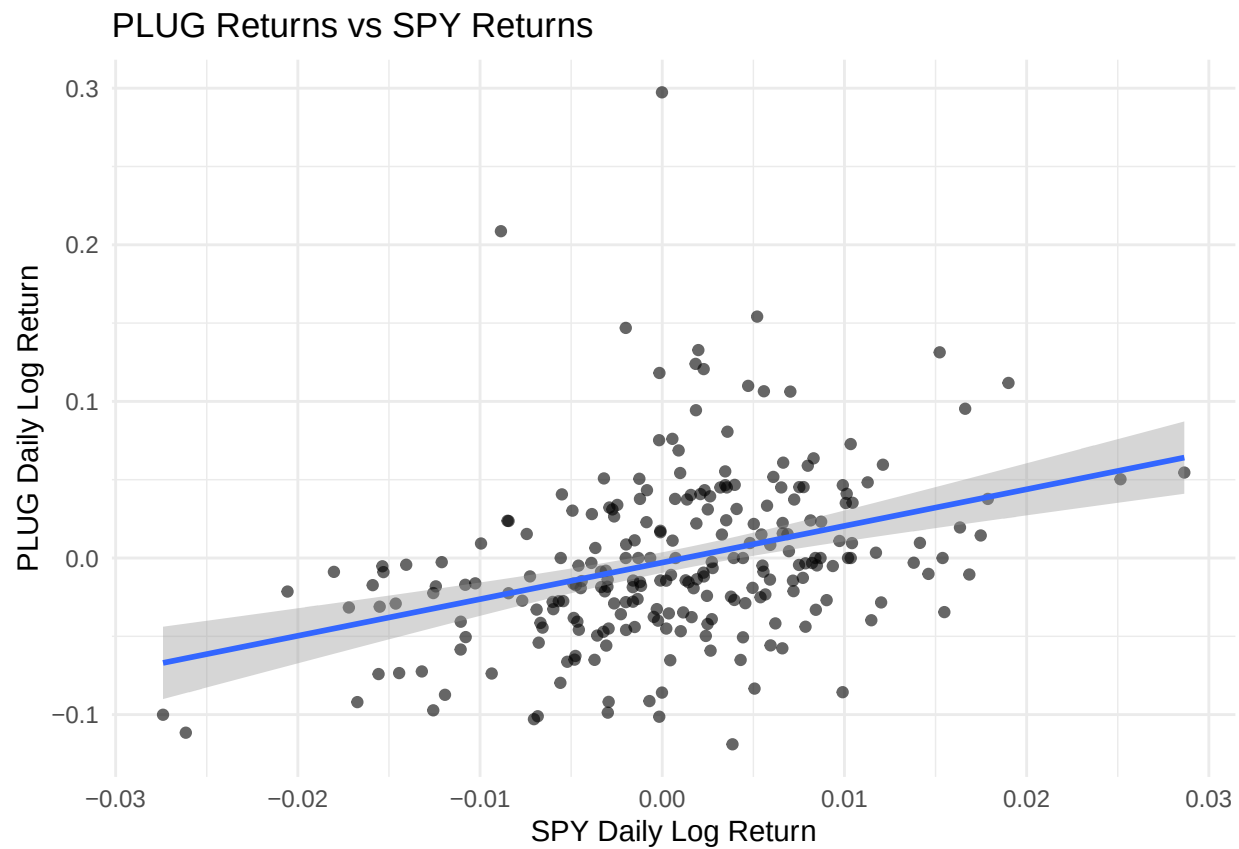


\$FNGR

FNGR Returns vs SPY Returns

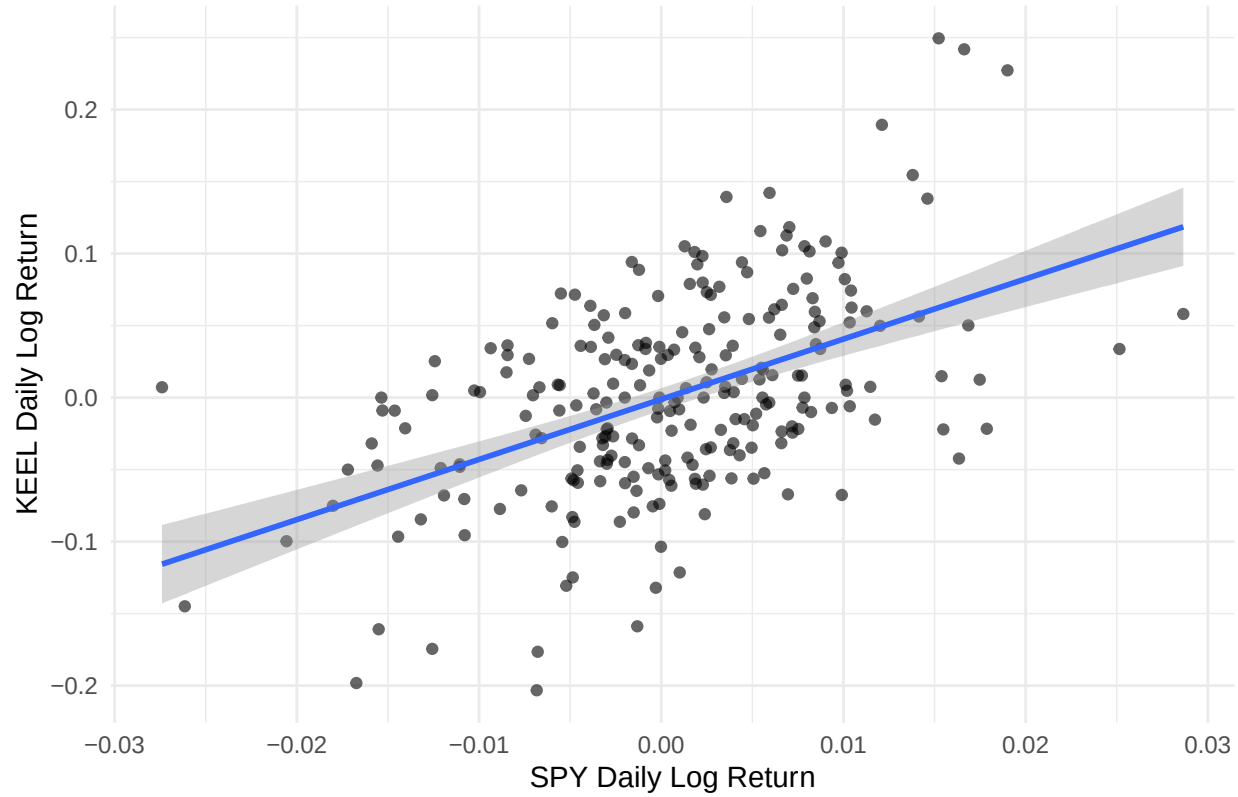


\$PLUG



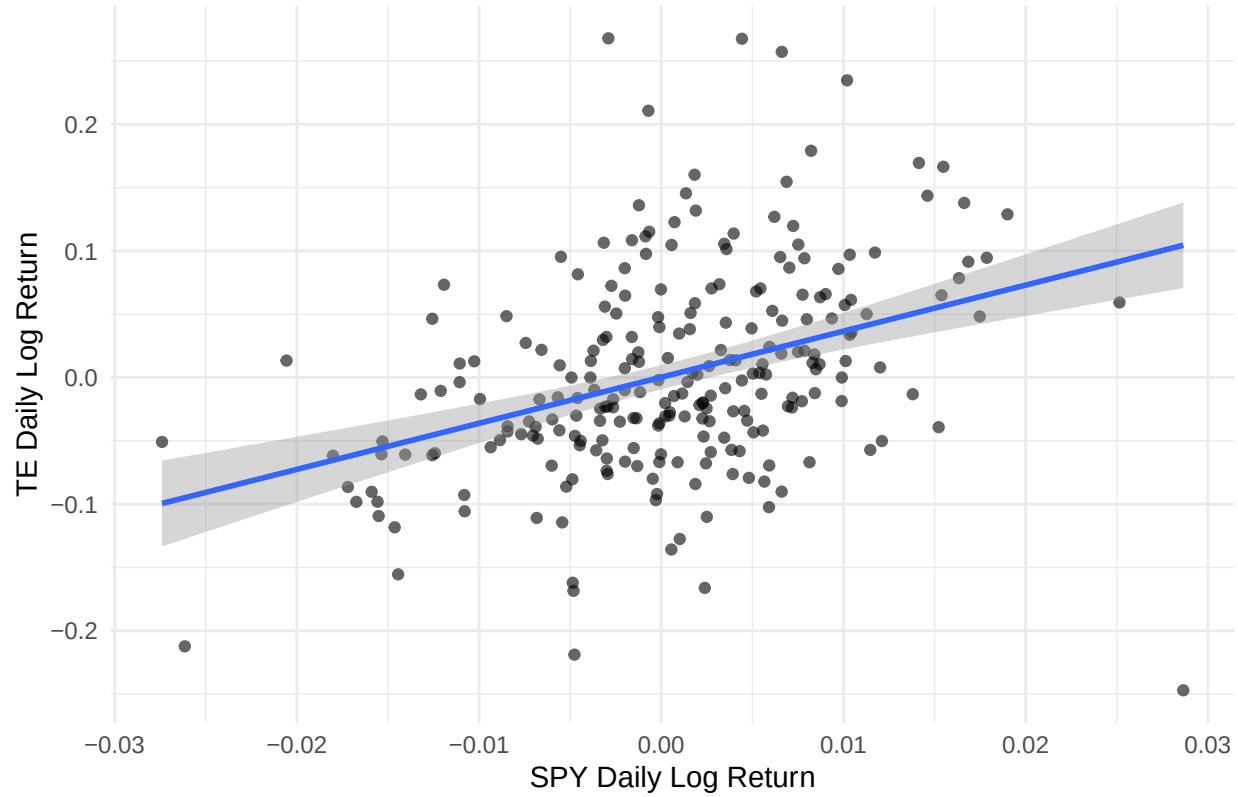
\$KEEL

KEEL Returns vs SPY Returns

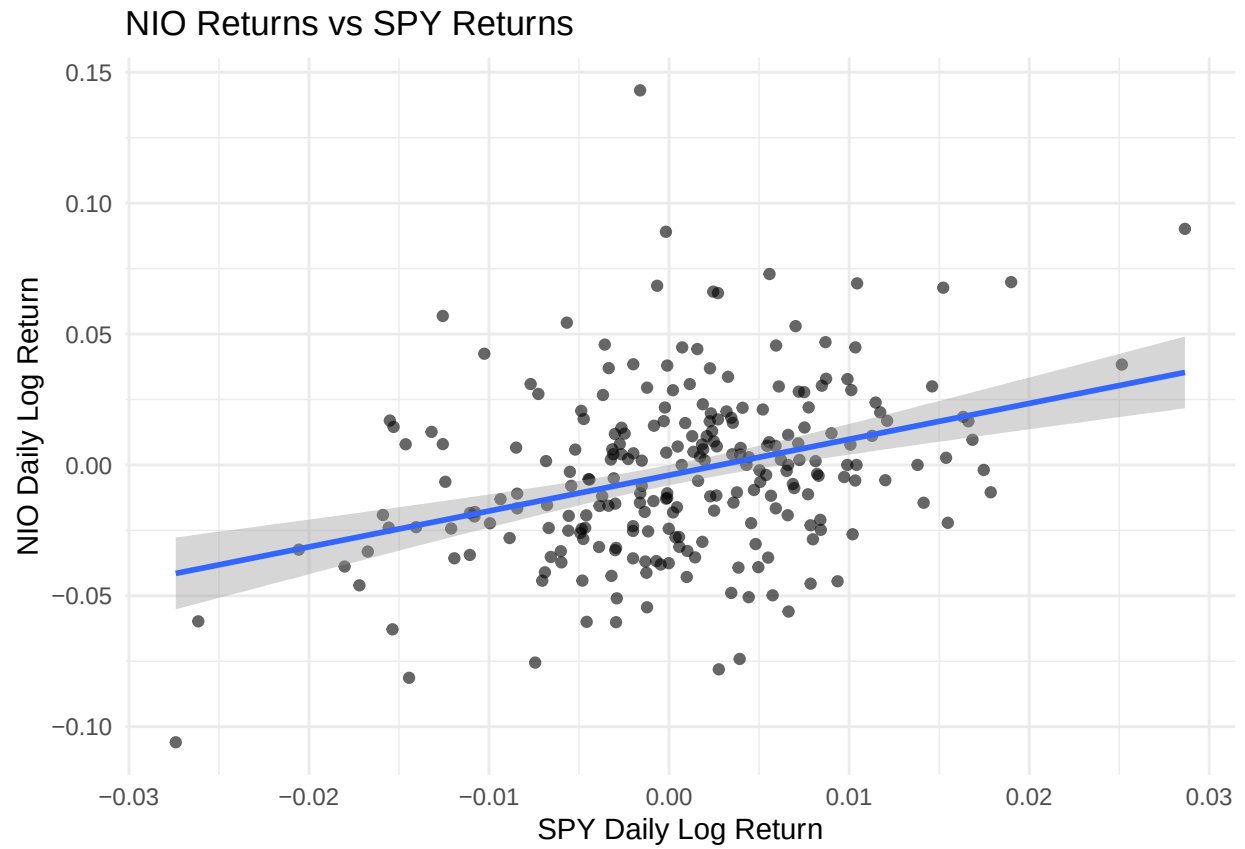


\$TE

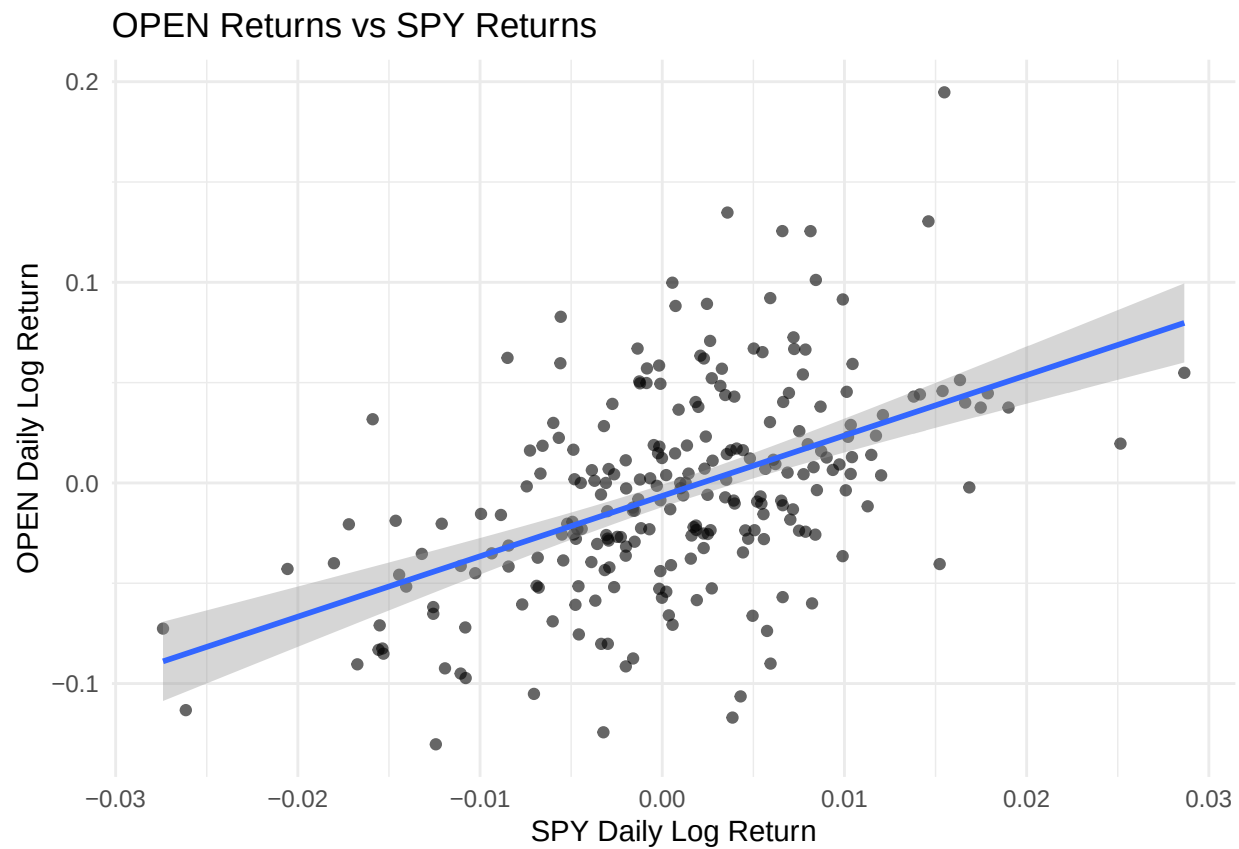
TE Returns vs SPY Returns



\$NIO

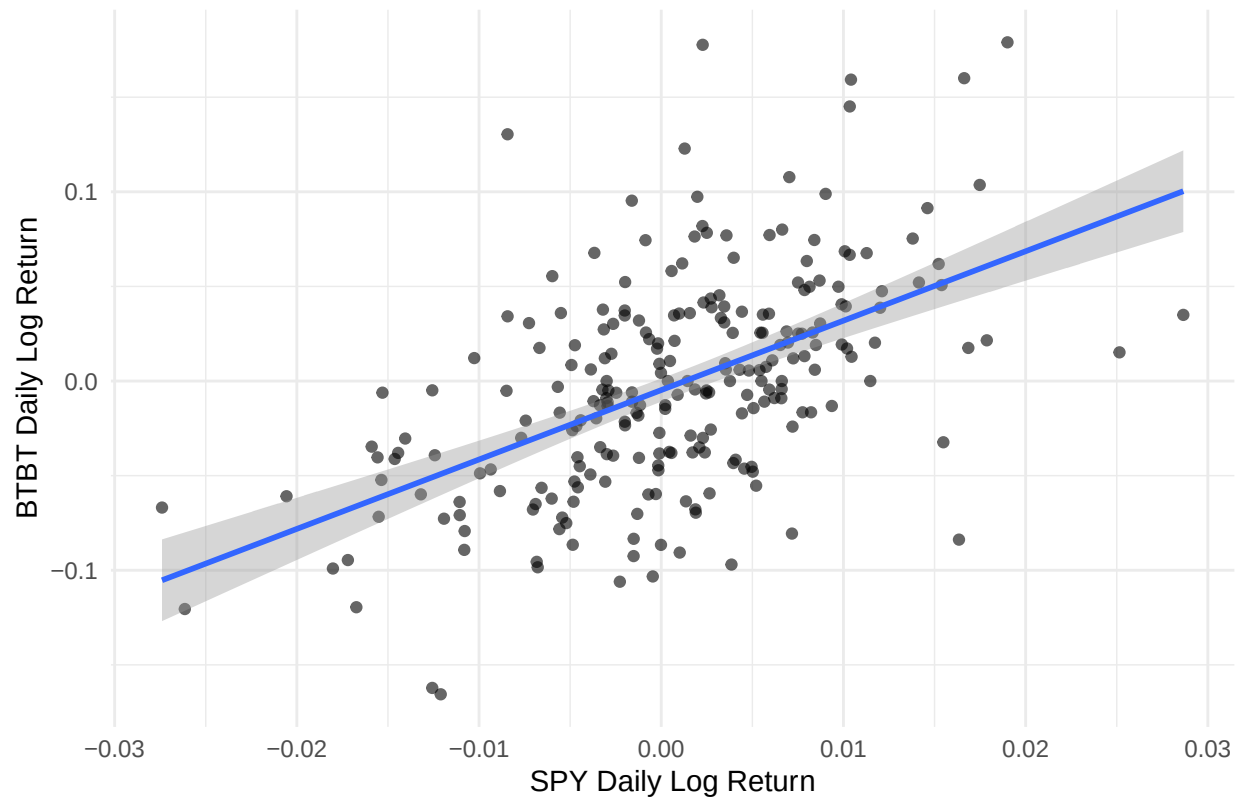


\$OPEN



\$BTBT

BTBT Returns vs SPY Returns



\$SPY

